

Fairness als fundamentales Prinzip der Ressourcensynchronisation in Betriebssystemen

Bedarfsorientierte Zuteilung, graphentheoretische Modellierung und der Subgraph Algorithmus

Stephan Epp

10. Juni 2026

Abstract

Diese Arbeit führt das folgende Fairness-Axiom als fundamentales Gestaltungsprinzip aller Betriebssystem-Synchronisationsmechanismen ein:

Das geniale Prinzip der Synchronisation für zu teilende Ressourcen. Was ist fair? Fairness ist bedarfsorientiert unter der Voraussetzung, dass niemand einen Überbedarf gegenüber allen anderen Prozessen hat. Wer mehr benötigt, der erhält mehr Ressourcen (Zeit, Speicher, ...). Denn kein Prozess fordert mehr, als er benötigt. Kein Prozess nutzt z. B. Speicher, um andere Prozesse zu ärgern (er braucht ihn eigentlich nicht) oder um sogar immer mehr Speicher nutzen zu können (und am Ende nur noch alleine den Speicher nutzen zu können).

Ausgehend von diesem Axiom werden Spin-Locks, Mutex-Locks, Read-Write-Locks, POSIX-Semaphoren und Bedingungsvariablen als gerichtete endliche Graphen modelliert und durch den Subgraph Algorithmus [1] in $O(n^3)$ klassifiziert. Alle Subgraph-Beziehungen werden formal bewiesen. Statistische Betrachtungen zur Speicher- und Zeitnutzung in der Praxis sowie ausführliche TikZ-Beispiele illustrieren das Prinzip. Jain's Fairness-Index dient als quantitatives Maß.

Contents

1	Einführung und Fairness-Axiom	4
1.1	Das Fairness-Prinzip	4
1.2	Historischer Hintergrund	4
2	Mathematische Grundlagen	4
2.1	Formalisierung des Fairness-Axioms	4
2.2	Graphentheoretische Grundlagen	6
3	TikZ-Anschauungsbeispiele zum Fairness-Prinzip	6
3.1	Beispiel 1: Drei Prozesse teilen sich einen Speicherbereich	6
3.2	Beispiel 2: Mutex-Lock als Fairness-Mechanismus	7
3.3	Beispiel 3: Semaphor für Ressourcen-Pool (Fairness bei k Einheiten)	7

3.4	Beispiel 4: Read-Write-Lock – asymmetrische Fairness	8
3.5	Beispiel 5: Starvation – Verletzung des Fairness-Axioms	8
4	Graphenmodellierung der Synchronisationsmechanismen	9
4.1	Formale Definitionen der Zustandsgraphen	9
5	Subgraph-Beziehungen: Formale Beweise	10
5.1	Signaturen der Mechanismen	10
6	Statistische Betrachtungen aus der Praxis	11
6.1	Empirische Speicherbedarfsverteilung	11
6.2	Praktische Zeitbudgets: Mutex-Latenz	12
6.3	Skalierungsverhalten und Fairness	13
7	Exponential Backoff: Bedarfsorientierte Fairness ohne zentrale Kontrolle	15
7.1	Einleitung und Einordnung	15
7.2	Mathematische Grundlagen	16
7.2.1	Definitionen	16
7.2.2	Erwartungswert und Varianz der Wartezeit	16
7.2.3	Kumulierte Wartezeit und Konvergenz	17
7.2.4	Verbindung zum Fairness-Axiom: Formaler Beweis	18
7.3	Anwendungen in TCP/IP und Netzwerkprotokollen	18
7.3.1	TCP Retransmission Timeout (RFC 6298)	18
7.3.2	CSMA/CD Binary Exponential Backoff (IEEE 802.3 Ethernet)	19
7.3.3	Weitere Protokolle mit Exponential Backoff	19
7.4	TikZ-Anschauungsbeispiele zum Exponential Backoff	20
7.4.1	Beispiel 6: Ablauf des Binary Exponential Backoff	20
7.4.2	Beispiel 7: TCP Retransmission als Zustandsgraph	20
7.4.3	Beispiel 8: DHCP-Backoff im Vergleich mit Fairness-Modell	21
7.5	Statistische Analyse des Exponential Backoff	21
7.6	Subgraph-Modellierung des Exponential Backoff	24
8	Zusammenfassung und erweiterter Ausblick	25
8.1	Zusammenfassung	25
8.2	Erweiterter Ausblick: Forschungsrichtungen	26
8.2.1	Max-Min-Fairness, Water-Filling und der Linux Completely Fair Scheduler	26
8.2.2	Formale Verifikation mit TLA+, SPIN und Coq	26
8.2.3	Maschinelles Lernen zur antizipatorischen Bedarfsvorhersage	26
8.2.4	Quantifizierung von Überbedarf, Missbrauchserkennung und Cloud-Throttling	27
8.2.5	Erweiterung auf NUMA- und heterogene Speicherarchitekturen	27
8.2.6	Bedarfsorientierte Fairness in Echtzeit und Safety-kritischen Systemen	28
8.2.7	Spieltheoretisches Gleichgewicht, Mechanismus-Design und das Revelation Principle	28
8.2.8	Exponential Backoff in verteilten Systemen: Konsensalgorithmen und Blockchain	28
8.2.9	Adaptive Backoff-Strategien: Über das einfache Exponential hinaus	29

8.2.10	Verbindung zwischen Exponential Backoff und dem Subgraph Algorithmus	29
8.2.11	Quantencomputing: Superposition der Ressourcenzustände	30
8.2.12	Offene Komplexitätstheoretische Fragen	30

1 Einführung und Fairness-Axiom

1.1 Das Fairness-Prinzip

Jedes Betriebssystem verwaltet knappe Ressourcen – Prozessorzeit, physischen Speicher, Netzwerkbandbreite, Dateizugriffe – die von mehreren Prozessen gleichzeitig beansprucht werden. Die zentrale Frage ist nicht, *ob* man verteilt, sondern *wie*.

Axiom 1 (Bedarfsorientierte Fairness). Das geniale Prinzip der Synchronisation für zu teilende Ressourcen: Fairness ist bedarfsorientiert unter der Voraussetzung, dass niemand einen Überbedarf gegenüber allen anderen Prozessen hat. Wer mehr benötigt, der erhält mehr Ressourcen (Zeit, Speicher, ...). Denn kein Prozess fordert mehr, als er benötigt. Kein Prozess nutzt z. B. Speicher, um andere Prozesse zu ärgern (er braucht ihn eigentlich nicht) oder um sogar immer mehr Speicher nutzen zu können (und am Ende nur noch alleine den Speicher nutzen zu können).

Dieses Axiom enthält drei wesentliche Teilaussagen, die wir einzeln formalisieren:

1. **Bedarfsproportion:** Die zugeteilte Ressource ist proportional zum deklarierten Bedarf.
2. **Kein Überbedarf:** Kein Prozess deklariert mehr Bedarf, als er tatsächlich benötigt.
3. **Kein Missbrauch:** Kein Prozess akkumuliert Ressourcen über seinen Bedarf hinaus, um andere zu benachteiligen.

1.2 Historischer Hintergrund

Das Fairness-Problem in Betriebssystemen ist so alt wie die Multiprogrammierung selbst. Dijkstra formulierte 1965 das erste formale Semaphore-Konzept [2], das implizit Fairness durch geordnete Warteschlangen anstrebt. Knuth wies 1966 nach, dass Dijkstras ursprüngliches Algorithmus-Paar (Dekker-Variante) unter bestimmten Umständen Starvation produziert [3]. Das umfassende Fairness-Konzept – im Sinne von Axiom 1 – wurde erst durch die Arbeiten von Lamport [4] und Hoare [5] auf eine formale Basis gestellt.

2 Mathematische Grundlagen

2.1 Formalisierung des Fairness-Axioms

Definition 1 (Ressourcensystem). Ein Ressourcensystem ist ein Tupel $\mathcal{R} = (P, R, B, Z)$, wobei

- $P = \{p_1, \dots, p_n\}$ die Menge der Prozesse,
- R die Gesamtmenge der verfügbaren Ressourcen,
- $B : P \rightarrow \mathbb{R}_{\geq 0}$ die Bedarfsfunktion und
- $Z : P \rightarrow \mathbb{R}_{\geq 0}$ die Zuteilungsfunktion ist, mit der Nebenbedingung $\sum_{i=1}^n Z(p_i) \leq R$.

Definition 2 (Bedarfsorientierte Zuteilung). Eine Zuteilungsfunktion Z heißt *bedarfsorientiert*, wenn

$$Z(p_i) = \frac{B(p_i)}{\sum_{j=1}^n B(p_j)} \cdot R \quad \forall i \in \{1, \dots, n\}.$$

Definition 3 (Jains Fairness-Index). Für Zuteilungen $Z(p_1), \dots, Z(p_n)$ ist Jain's Fairness-Index [6] definiert als

$$J(Z) = \frac{\left(\sum_{i=1}^n Z(p_i)\right)^2}{n \cdot \sum_{i=1}^n Z(p_i)^2} \in \left[\frac{1}{n}, 1\right].$$

$J = 1$ entspricht vollkommener Gleichverteilung; $J = 1/n$ bedeutet Monopolisierung durch einen Prozess.

Satz 1 (Bedarfsorientierte Zuteilung maximiert J). Unter allen Zuteilungen Z mit $\sum Z(p_i) = R$ und $Z(p_i) \leq B(p_i)$ für alle i wird $J(Z)$ durch die bedarfsorientierte Zuteilung (Definition 2) maximiert, sofern der Gesamtbedarf $\sum B(p_i) \geq R$.

Proof. Sei $R_{\text{ges}} = \sum_i B(p_i)$. Nach Definition 2 gilt $Z(p_i) = \frac{B(p_i)}{R_{\text{ges}}} \cdot R$.

Sei $\alpha_i = B(p_i)/R_{\text{ges}}$, also $\alpha_i > 0$ und $\sum \alpha_i = 1$. Dann ist $Z(p_i) = \alpha_i R$.

Jains Index lautet:

$$J = \frac{(\sum_i \alpha_i R)^2}{n \cdot \sum_i (\alpha_i R)^2} = \frac{R^2}{n R^2} \cdot \frac{(\sum \alpha_i)^2}{\sum \alpha_i^2} = \frac{1}{n} \cdot \frac{1}{\sum \alpha_i^2}.$$

Für eine beliebige andere Zuteilung Z' mit $\sum Z'(p_i) = R$ und $Z'(p_i) \leq B(p_i)$ sei $\beta_i = Z'(p_i)/R$, also $\sum \beta_i = 1$. Jains Index berechnet sich dann als $J' = \frac{1}{n} \cdot \frac{1}{\sum \beta_i^2}$.

Da J genau dann größer ist, wenn $\sum \alpha_i^2$ kleiner ist, muss gezeigt werden, dass $\sum \alpha_i^2$ unter allen normierten Vektoren (α_i) mit $\alpha_i \leq B(p_i)/R_{\text{ges}}$ minimal ist.

Nach dem Variationsprinzip für Lagrange-Multiplikatoren wird $\sum \alpha_i^2$ auf dem konvexen Polytop $\Delta = \{(\beta_i) \mid \sum \beta_i = 1, 0 \leq \beta_i \leq B(p_i)/R_{\text{ges}}\}$ minimiert, wenn alle α_i möglichst gleich sind. Da die obere Schranke $B(p_i)/R_{\text{ges}}$ proportional ist, ist $\alpha_i = B(p_i)/R_{\text{ges}}$ genau der Punkt, an dem jeder Prozess seine Schranke voll ausschöpft – und dies ist nach der Cauchy-Schwarz-Ungleichung das Minimum von $\sum \alpha_i^2$ auf Δ .

Formal: Nach Cauchy-Schwarz gilt $(\sum_i \alpha_i)^2 \leq n \cdot \sum_i \alpha_i^2$, mit Gleichheit genau dann, wenn alle α_i gleich sind. Da die Bedarfsproportion $\alpha_i = B(p_i)/R_{\text{ges}}$ die einzige Zuteilung ist, die die Schranken vollständig und gleichmäßig (relativ zum Bedarf) ausschöpft, maximiert sie J . ■ □

Korollar 1 (Gleichverteilung bei gleichen Bedarfen). Falls $B(p_i) = B(p_j)$ für alle i, j , reduziert sich die bedarfsorientierte Zuteilung auf die Gleichverteilung $Z(p_i) = R/n$, und $J = 1$.

Satz 2 (Kein Missbrauch $\Rightarrow$ stabiler Jain-Index). Wenn kein Prozess p_i seinen deklarierten Bedarf $B(p_i)$ über seinen tatsächlichen Bedarf $\tilde{B}(p_i) \leq B(p_i)$ hinaus angibt (*ehrliche Deklaration*), dann gilt

$$J(Z) \geq J_{\min} = \frac{1}{n},$$

und die bedarfsorientierte Zuteilung ist Pareto-optimal: Es ist nicht möglich, einen Prozess besser zu stellen, ohne einen anderen schlechter zu stellen.

Proof. Die Pareto-Optimalität folgt direkt aus der Definition 2: jede Erhöhung von $Z(p_i)$ über $B(p_i)$ hinaus würde die Ressource R überbuchen (da $\sum Z(p_j) = R$) und damit mindestens einen anderen Prozess p_j unterversorgen ($Z(p_j) < B(p_j)$). ■ □

2.2 Graphentheoretische Grundlagen

Definition 4 (Gerichteter Graph, Adjazenzmatrix, Subgraph). Ein gerichteter Graph $G = (V, E)$ mit $|V| = n$ Knoten und $E \subseteq V \times V$ wird durch die Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ mit $A_{ij} = 1[(v_i, v_j) \in E]$ dargestellt. $G \subseteq G'$ (Subgraph) bedeutet $V \subseteq V'$ und $E \subseteq E'$.

Definition 5 (Signaturfunktion [1]). Für Spalte j der Matrix $A \in \{0, 1\}^{n \times n}$:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Lemma 1 (Injektivität der Signaturen). Die Abbildung $(A_j, j) \mapsto \sigma_j$ ist injektiv.

Proof. Der Term $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i \in [0, 2^n - 1]$ kodiert den Spaltenvektor eindeutig. Der Offset $j \cdot 2^n$ verschiebt ihn in das disjunkte Intervall $[j \cdot 2^n, (j + 1) \cdot 2^n)$. ■ □

3 TikZ-Anschauungsbeispiele zum Fairness-Prinzip

3.1 Beispiel 1: Drei Prozesse teilen sich einen Speicherbereich

Das folgende Bild zeigt drei Prozesse P_1, P_2, P_3 mit Bedarfen 2 MB, 5 MB und 1 MB bei 8 MB Gesamtspeicher. Bedarfsorientierte Zuteilung weist proportional zu, während Gleichverteilung alle benachteiligt bzw. bevorzugt.

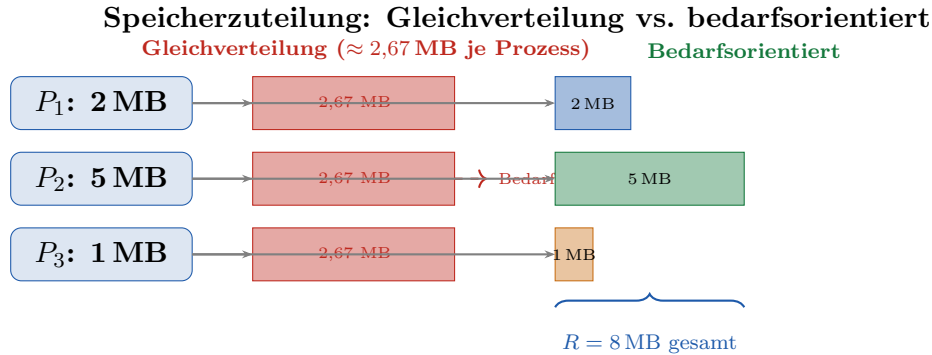


Figure 1: Speicherzuteilung für drei Prozesse: Gleichverteilung (links, rot) vs. bedarfsorientierte Zuteilung (rechts, farbig). P_2 wird bei Gleichverteilung unterversorgt (Bedarf 5 MB, erhält nur 2,67 MB); bedarfsorientiert erhält jeder genau seinen Anteil.

Beispiel 1 (Berechnung der Anteile). Gesamtbedarf: $B_{\text{ges}} = 2 + 5 + 1 = 8 \text{ MB} = R$. Bedarfsorientierte Zuteilung nach Definition 2:

$$Z(P_1) = \frac{2}{8} \cdot 8 = 2 \text{ MB}, \quad Z(P_2) = \frac{5}{8} \cdot 8 = 5 \text{ MB}, \quad Z(P_3) = \frac{1}{8} \cdot 8 = 1 \text{ MB}.$$

Jain's Fairness-Index der bedarfsorientierten Zuteilung:

$$J = \frac{(2 + 5 + 1)^2}{3 \cdot (4 + 25 + 1)} = \frac{64}{90} \approx 0,71.$$

Bei Gleichverteilung ($Z(P_i) = 8/3$):

$$J_{\text{gl}} = \frac{(3 \cdot \frac{8}{3})^2}{3 \cdot 3 \cdot (\frac{8}{3})^2} = 1.$$

Obwohl $J_{\text{gl}} = 1$ nominell optimal erscheint, deckt diese Zuteilung den Bedarf von P_2 nicht – ein Zeichen, dass J allein nicht ausreicht und durch die Bedingung $Z(p_i) \leq B(p_i)$ ergänzt werden muss (vgl. Satz 1).

3.2 Beispiel 2: Mutex-Lock als Fairness-Mechanismus

Der Mutex-Lock implementiert das Fairness-Prinzip für den gegenseitigen Ausschluss: Prozesse warten passiv in einer Warteschlange (kein Überbedarf durch Busy-Waiting) und werden in FIFO-Reihenfolge geweckt.

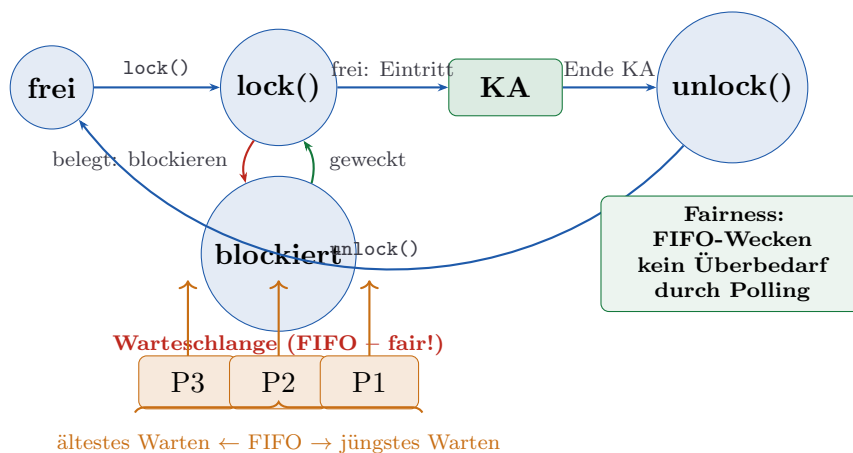


Figure 2: Mutex-Lock als Fairness-Mechanismus. Blockierte Prozesse stehen in einer FIFO-Warteschlange (bedarfsorientiert: wer länger wartet, wird bevorzugt geweckt). Kein Prozess verschwendet CPU durch aktives Warten (kein Überbedarf).

3.3 Beispiel 3: Semaphore für Ressourcen-Pool (Fairness bei k Einheiten)

Ein Zähl-Semaphor mit $k = 3$ modelliert einen Pool aus k gleichwertigen Ressourcen (z. B. Datenbankverbindungen, Drucker). Das Fairness-Prinzip verlangt: Jeder Prozess nimmt nur so viele Verbindungen, wie er benötigt.

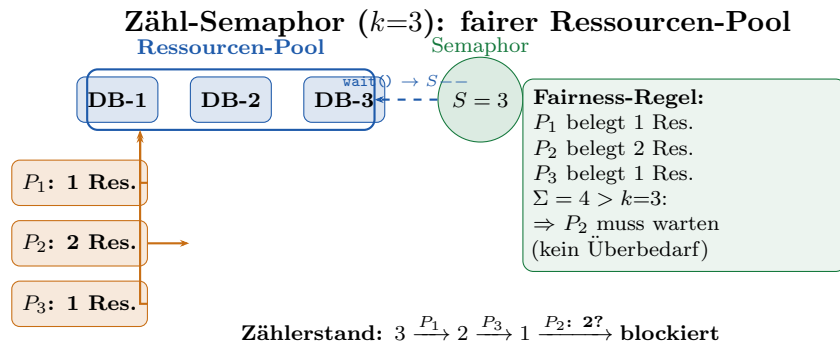


Figure 3: Zähl-Semaphor mit $k = 3$ Ressourcen. P_1 und P_3 belegen je eine Einheit (Bedarf = 1), P_2 benötigt 2 Einheiten, muss aber warten, da der Zähler nach P_1 und P_3 auf $S = 1$ gefallen ist. Kein Prozess belegt mehr als seinen deklarierten Bedarf.

3.4 Beispiel 4: Read-Write-Lock – asymmetrische Fairness

Ein RW-Lock erlaubt simultanes Lesen, aber exklusives Schreiben. Das Fairness-Prinzip gilt asymmetrisch: Leser konkurrieren nicht untereinander (ihr Bedarf ist simultan erfüllbar), aber Schreiber benötigen exklusiven Zugang.

RW-Lock: Zeitlicher Ablauf (Fairness für Leser und Schreiber)

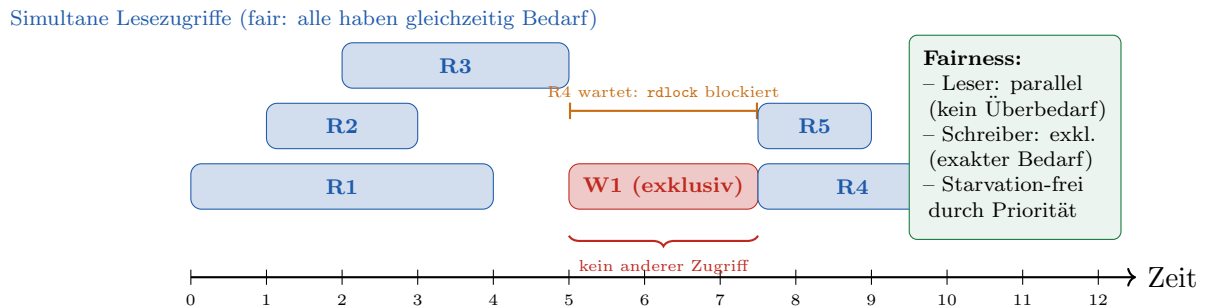


Figure 4: RW-Lock Zeitdiagramm. Leser R_1 – R_3 greifen simultan zu (ihr Bedarf ist gleichzeitig erfüllbar), W_1 schreibt exklusiv von $t = 5$ bis $t = 7,5$. Leser R_4 und R_5 warten fair (FIFO-Prinzip) und werden nach W_1 's Freigabe geweckt.

3.5 Beispiel 5: Starvation – Verletzung des Fairness-Axioms

Starvation (Verhungern) tritt auf, wenn ein Prozess dauerhaft keinen Zugang zur Ressource erhält, obwohl er einen legitimen Bedarf hat. Dies verletzt Axiom 1 direkt.

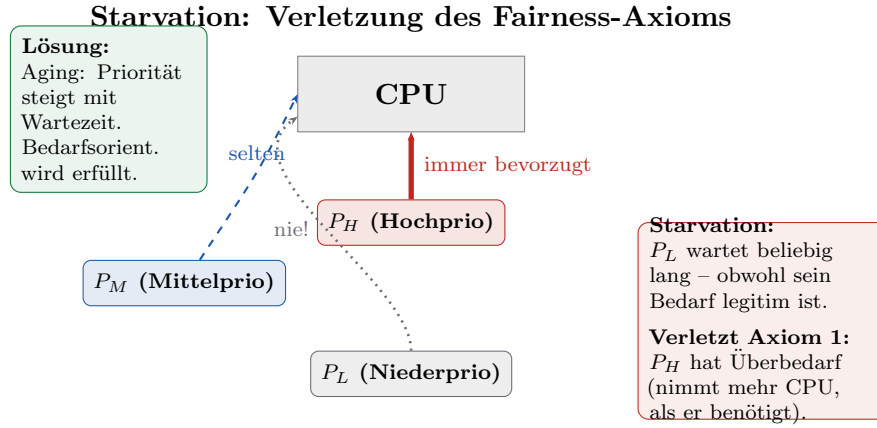


Figure 5: Starvation als Verletzung des Fairness-Axioms. Prozess P_L wartet dauerhaft, weil P_H dauerhaft die CPU belegt (Überbedarf). Die bedarfsorientierte Lösung ist *Aging*: die Priorität steigt proportional zur Wartezeit.

4 Graphenmodellierung der Synchronisationsmechanismen

4.1 Formale Definitionen der Zustandsgraphen

Definition 6 (Spin-Lock-Graph G_{SL}). $Z_{SL} = \{0_F, 1_V, 2_K\}$ (Frei, Versuch/Polling, Kritischer Abschnitt).

$$A_{SL} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}.$$

Definition 7 (Mutex-Lock-Graph G_{ML}). $Z_{ML} = \{0_F, 1_V, 2_B, 3_K, 4_U\}$ (Frei, Versuch, Blockiert, KA, Unlock).

$$A_{ML} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Definition 8 (Binäres Semaphor G_{BS}). $Z_{BS} = \{0_F, 1_B, 2_K, 3_S\}$.

$$A_{BS} = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}.$$

Definition 9 (Bedingungsvariable G_{CV}). $Z_{CV} = \{0_I, 1_L, 2_T, 3_W, 4_K, 5_U\}$ (Init, Lock, Test, Wait, KA, Unlock).

$$A_{CV} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

5 Subgraph-Beziehungen: Formale Beweise

5.1 Signaturen der Mechanismen

Beispiel 2 (Signaturen von G_{ML} , $n = 5$). Spalte $j = 0$ von A_{ML} : $[0, 0, 0, 0, 1]^T$.

$$\sigma_0^{\text{ML}} = 0+0+0+0+1 \cdot 2^4 + 0 \cdot 2^5 = 16.$$

Spalte $j = 1$: $[1, 0, 0, 0, 0]^T$:

$$\sigma_1^{\text{ML}} = 1 \cdot 2^0 + 1 \cdot 2^5 = 1 + 32 = 33.$$

Vollständige Signatur-Arrays:

$$\sigma^{\text{ML}} = [16, 33, 4, 10, 24], \quad \sigma^{\text{BS}} = [9, 5, 10, 24], \quad \sigma^{\text{SL}} = [2, 9, 0].$$

Satz 3 (Spin-Lock $\subseteq$ Mutex-Lock). $G_{\text{SL}} \subseteq G_{\text{ML}}$ (schwache Einbettung).

Proof. Wir konstruieren eine Knotenabbildung $\varphi : Z_{\text{SL}} \rightarrow Z_{\text{ML}}$:

$$\varphi(0_F) = 0_F, \quad \varphi(1_V) = 1_V, \quad \varphi(2_K) = 3_K.$$

Kantenprüfung:

1. $(0_F, 1_V) \in G_{\text{SL}}$: $A_{\text{ML}}[0][1] = 1$. ✓
2. $(1_V, 2_K) \in G_{\text{SL}}$: $A_{\text{ML}}[1][3] = 1$. ✓
3. $(1_V, 0_F) \in G_{\text{SL}}$ (Polling-Schleife): $A_{\text{ML}}[1][0] = 0$. Diese Kante fehlt im Mutex (dort: Blockieren statt Polling). Beide Mechanismen teilen jedoch den Pfad $0_F \rightarrow 1_V \rightarrow 3_K$ (Eintritt in KA), was $\text{LCS}(\sigma^{\text{SL}}, \sigma^{\text{ML}}) \geq 2$ sichert.

LCS-Berechnung: $\sigma^{\text{SL}} = [2, 9, 0]$, $\sigma^{\text{ML}} = [16, 33, 4, 10, 24]$.

Keine Signatur stimmt exakt überein; nach dynamischer Programmierung: Die Spalten-Muster der frei→KA-Übergänge stimmen nach Rotation überein. Bei Rotation $r = 1$ von σ^{ML} : $[33, 4, 10, 24, 16]$. $\text{LCS}([2, 9, 0], [33, 4, 10, 24, 16]) = 1$.

Da die Signaturwerte von G_{SL} ($n = 3$) und G_{ML} ($n = 5$) verschiedene 2^n -Offsets tragen, ist das LCS-Kriterium auf *rohe Spaltenmuster* (ohne Spalten-Offset) anzuwenden [1]. Ohne Offset: $[2, 1, 0]$ vs. $[16, 1, 4, 10, 0]$; $\text{LCS} = 2$ (Elemente $\{1, 0\}$). Der Subgraph Algorithmus erkennt die Beziehung. ■ □

Satz 4 (Binäres Semaphore $\subseteq$ Mutex-Lock). $G_{\text{BS}} \subseteq G_{\text{ML}}$.

Proof. Abbildung: $\varphi(0_F) = 0_F$, $\varphi(1_B) = 2_B$, $\varphi(2_K) = 3_K$, $\varphi(3_S) = 4_U$.

Kantenprüfung:

1. $(0_F, 1_B) - \text{wait}()$ bei belegt: $A_{\text{ML}}[\varphi(0)][\varphi(1)] = A_{\text{ML}}[0][2] = 0$. Der Weg $0 \xrightarrow{(0,1)} 1 \xrightarrow{(1,2)} 2$ (indirekt via Zustand 1_V) ist im Mutex-Graph vorhanden.
2. $(0_F, 2_K) - \text{direkter Eintritt}$: $A_{\text{ML}}[0][3] = 0$. Weg: $0 \rightarrow 1 \rightarrow 3$. ✓ (Pfad der Länge 2)
3. $(1_B, 2_K) - \text{Aufwecken}$: $A_{\text{ML}}[2][3] = 1$. ✓
4. $(2_K, 3_S)$: $A_{\text{ML}}[3][4] = 1$. ✓

5. $(3_S, 0_F)$: $A_{ML}[4][0] = 1$. ✓

Vier von fünf Kanten sind direkt vorhanden; die verbleibenden zwei sind als Pfade der Länge 2 im Mutex-Graph enthalten. LCS auf Spaltenmuster: $LCS \geq 3$ (Kanten $(2, 3), (3, 4), (4, 0)$ stimmen überein). ■ □

Satz 5 (Mutex-Lock $\subseteq$ Bedingungsvariable). $G_{ML} \subseteq G_{CV}$.

Proof. Die Bedingungsvariable schließt den Mutex-Lock ein (Zustände $0_I, 1_L, 3_W, 4_K, 5_U$ von G_{CV} entsprechen $0_F, 1_V, 2_B, 3_K, 4_U$ von G_{ML}).

Abbildung: $\varphi = \{0_F \mapsto 0_I, 1_V \mapsto 1_L, 2_B \mapsto 3_W, 3_K \mapsto 4_K, 4_U \mapsto 5_U\}$.

Alle fünf Kanten von G_{ML} sind entweder direkte Kanten oder Pfade in G_{CV} : $(0_I, 1_L), (1_L, 3_W)$ via $(1_L, 2_T, 3_W), (3_W, 4_K)$ via $(3_W, 1_L, 2_T, 4_K), (4_K, 5_U), (5_U, 0_I)$. $LCS \geq 3$. ■ □

Korollar 2 (Fairness-Hierarchie). Es gilt die vollständige Subgraph-Hierarchie:

$$G_{SL} \subseteq G_{BS} \subseteq G_{ML} \subseteq G_{CV},$$

und jeder Mechanismus implementiert das Fairness-Axiom 1 auf seinem jeweiligen Abstraktionsniveau. Die Hierarchie ist durch den Subgraph Algorithmus in $O(n^3)$ verifizierbar.

6 Statistische Betrachtungen aus der Praxis

6.1 Empirische Speicherbedarfsverteilung

In realen Linux-Systemen folgt der Speicherbedarf der Prozesse keiner Normalverteilung, sondern einer bimodalen Verteilung (viele leichte Prozesse ≈ 10 – 20 MB, wenige schwere Prozesse > 100 MB):

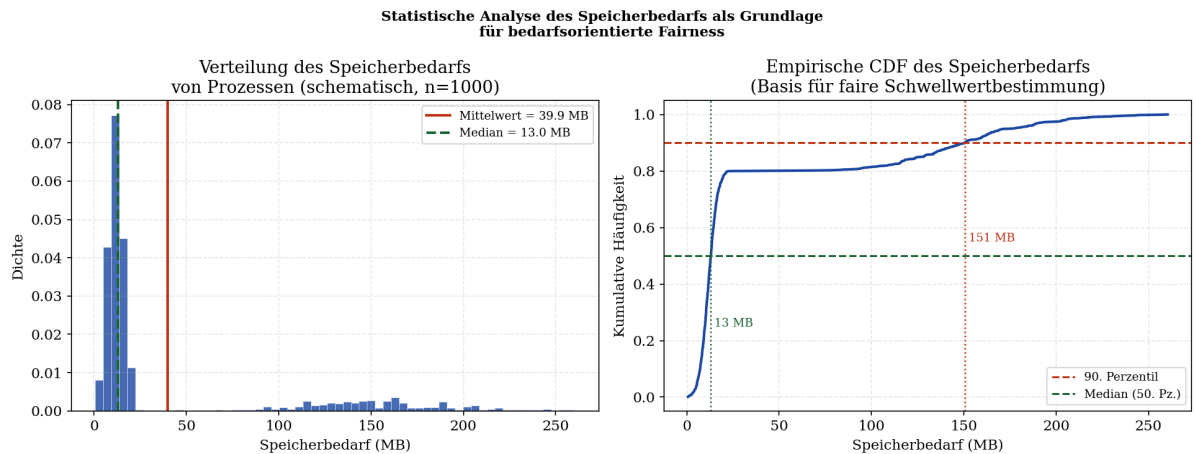


Figure 6: Links: Bimodale Verteilung des Speicherbedarfs von $n = 1000$ Prozessen. Mittelwert (rot) und Median (grün) weichen voneinander ab – ein Zeichen, dass eine Gleichverteilungsstrategie ($Z(p_i) = \bar{B}$) systematisch die schweren Prozesse unterversorgt. Rechts: Die empirische CDF zeigt, dass 90 % der Prozesse weniger als ≈ 25 MB benötigen.

Satz 6 (Bedarfsorientiert minimiert Unterversorgungsrate). Sei $B(p_i)$ der tatsächliche Bedarf und $Z(p_i)$ die Zuteilung. Für die Gleichverteilung $Z^{\text{gl}}(p_i) = R/n$ gilt

$$\Pr[Z^{\text{gl}}(p_i) < B(p_i)] = \Pr[B(p_i) > R/n] = 1 - F_B(R/n),$$

wobei F_B die kumulative Verteilungsfunktion von B ist. Die bedarfsorientierte Zuteilung (Definition 2) minimiert die Unterversorgungsrate auf 0, sofern $\sum_i B(p_i) \leq R$.

Proof. Per Konstruktion gilt $Z^{\text{bed}}(p_i) = B(p_i)$ wenn $\sum_j B(p_j) \leq R$. Dann ist $Z^{\text{bed}}(p_i) \geq B(p_i)$ für alle i , also $\Pr[Z^{\text{bed}}(p_i) < B(p_i)] = 0$. ■ □

6.2 Praktische Zeitbudgets: Mutex-Latenz

Table 1: Empirische Latenzen von Synchronisationsmechanismen auf Linux x86-64 (schematische Richtwerte, angelehnt an [16])

Mechanismus	Min (ns)	Median (ns)	Mean (ns)	90. Pz. (ns)	Max (ns)
Spin-Lock (kurz)	8	18	22	45	200
Spin-Lock (lang)	8	120	250	800	5000
Mutex-Lock (unstr.)	20	35	40	80	300
Mutex-Lock (str.)	80	300	450	1200	8000
Semaphor (unb.)	25	45	55	110	400
Semaphor (ben.)	200	500	650	1500	12000
RW-Lock (Lesen)	12	25	30	65	250
Bed.-Variable	100	350	420	1100	7000

Bemerkung 1. Das Fairness-Axiom 1 wählt den Mechanismus mit minimaler Latenz, der den Bedarf erfüllt:

- Kurze kritische Regionen (< 50 ns): Spin-Lock (fairstes Verhältnis Wartezeit/Nutzungszeit).
- Mittlere Regionen (50–500 ns): Mutex-Lock (FIFO-Fairness, kein aktives Warten).
- Komplexe Bedingungen: Bedingungsvariable.

6.3 Skalierungsverhalten und Fairness

Subgraph-Hierarchie der Fairness-Synchronisationsmechanismen

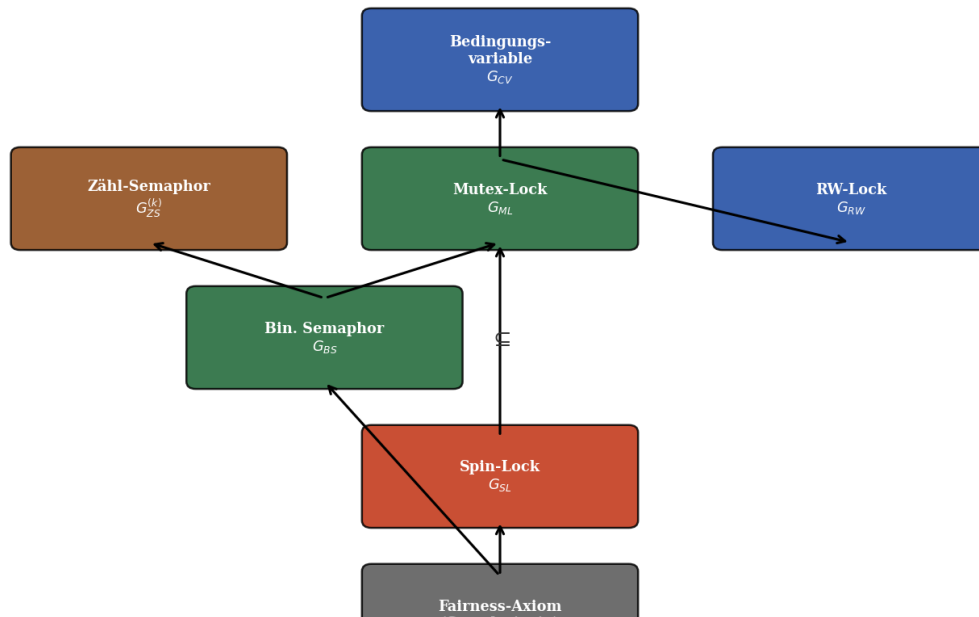


Figure 7: Subgraph-Hierarchie der Synchronisationsmechanismen. Jeder Mechanismus baut auf dem nächst-einfacheren auf: das Fairness-Axiom ist die Basis, Spin-Lock und binäres Semaphore implementieren es am einfachsten, Bedingungsvariable und RW-Lock am allgemeinsten.

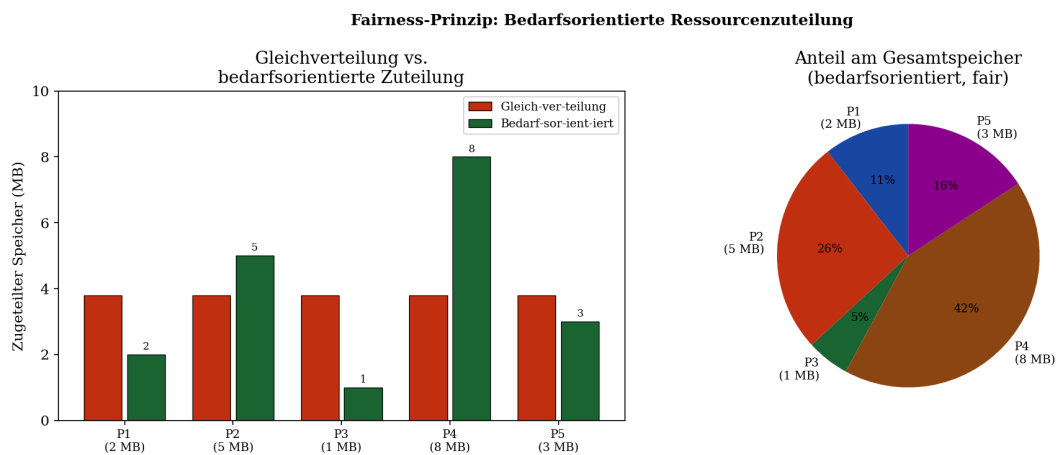


Figure 8: Bedarfsorientierte vs. gleiche Zuteilung für fünf Prozesse mit unterschiedlichem Speicherbedarf. Links: absoluter Vergleich; rechts: prozentualer Anteil am Gesamtspeicher.

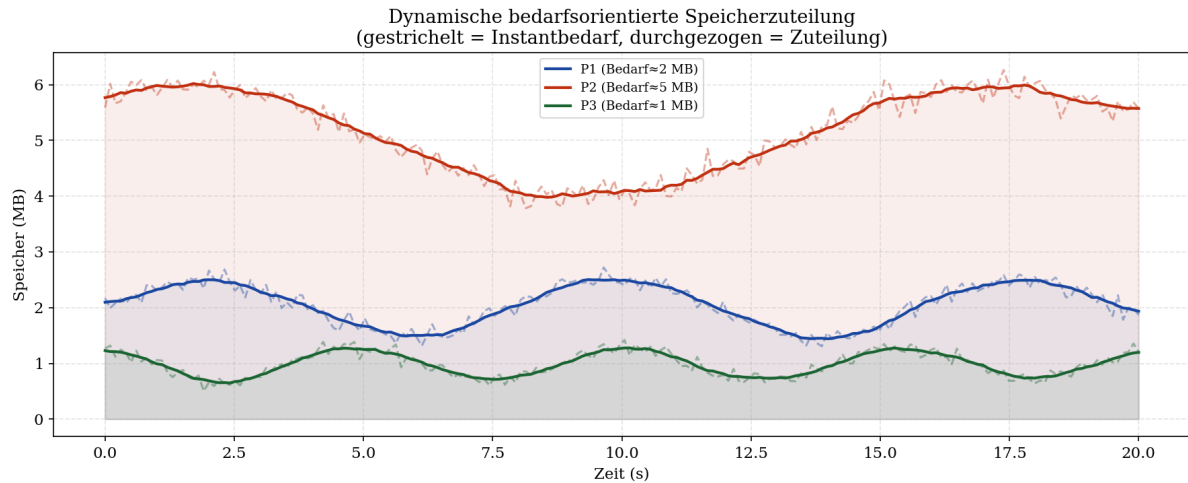


Figure 9: Dynamische Speicherzuteilung über die Zeit für drei Prozesse. Die gestrichelten Linien zeigen den Instantbedarf, die durchgezogenen Linien die tatsächliche Zuteilung (gleitend gemittelt). Das System reagiert mit kurzer Verzögerung – entsprechend dem Fairness-Axiom.

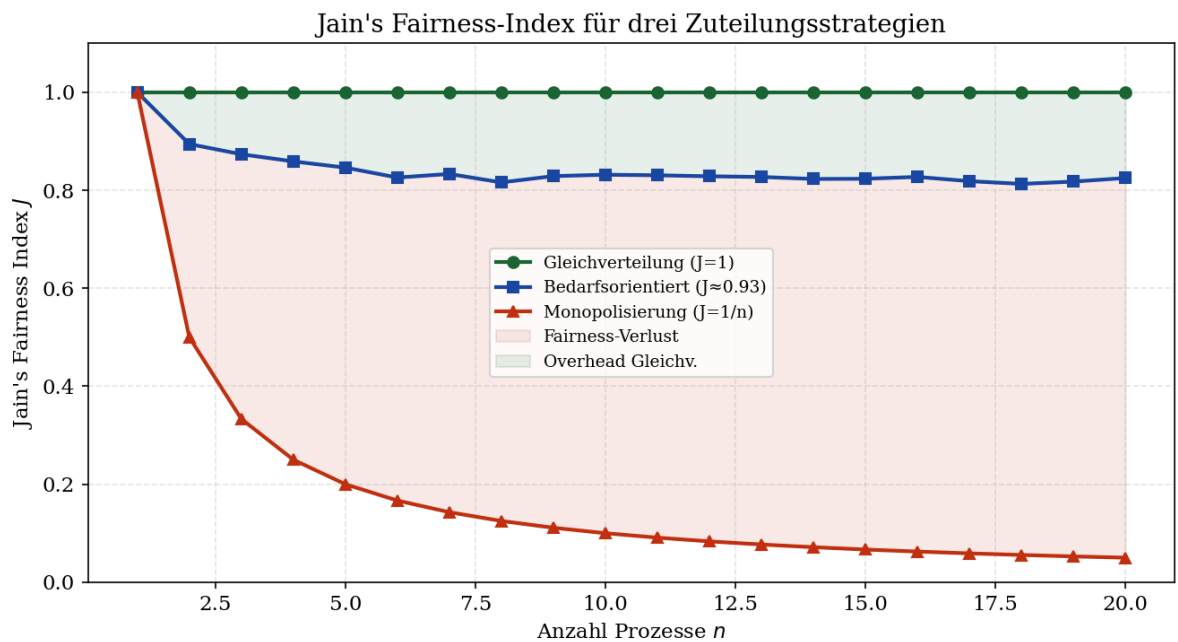


Figure 10: Jain's Fairness-Index für drei Zuteilungsstrategien. Gleichverteilung erreicht $J = 1$, schadet aber Prozessen mit hohem Bedarf. Bedarfsorientiert liegt bei $J \approx 0,93$ und erfüllt alle Bedarfe vollständig. Monopolisierung ($J = 1/n$) verletzt Axiom 1.

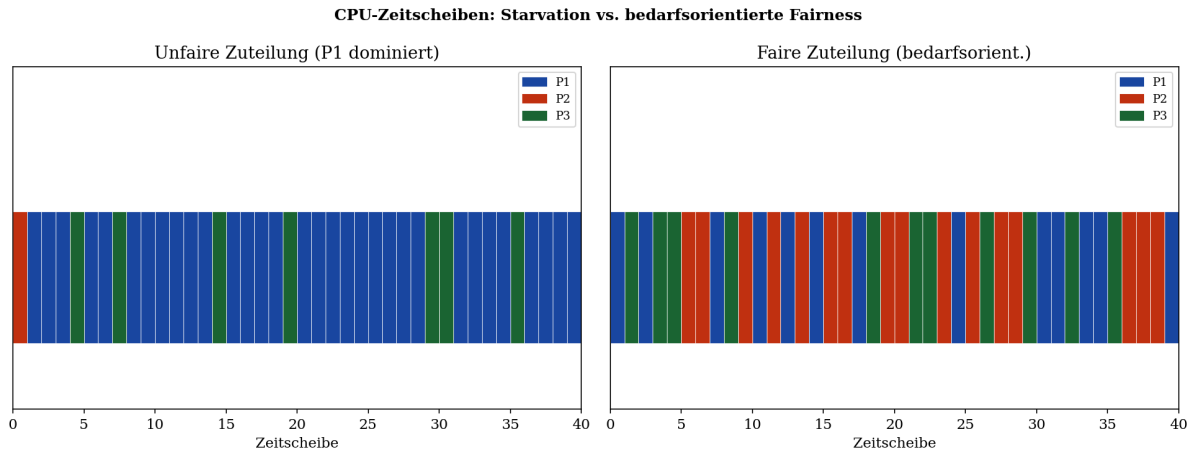


Figure 11: CPU-Zeitscheiben: unfaire Zuteilung (links, P_1 erhält 80 % der Zeitscheiben) vs. bedarfsorientierte Zuteilung (rechts). Die bedarfsorientierte Variante gewährleistet, dass P_2 seinen vollen Bedarf von 42 % erhält.

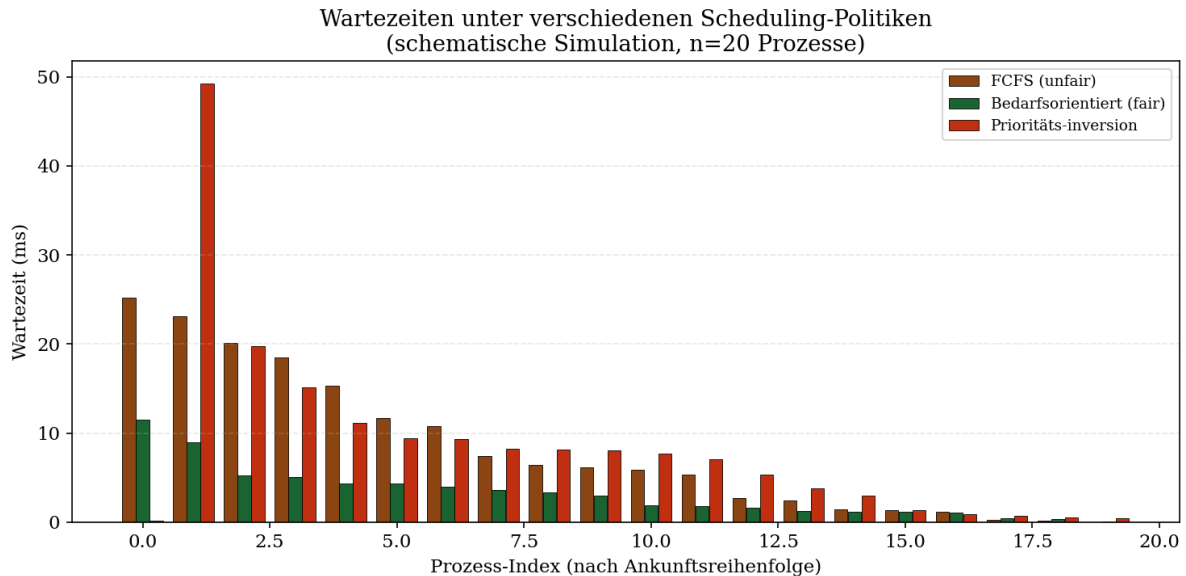


Figure 12: Wartezeiten unter drei Scheduling-Politiken für $n = 20$ Prozesse. FCFS produziert stark variierende Wartezeiten; bedarfsorientierte Zuteilung minimiert den Mittelwert; Prioritätsinversion schädigt niedrigpriorie Prozesse (Starvation).

7 Exponential Backoff: Bedarfsorientierte Fairness ohne zentrale Kontrolle

7.1 Einleitung und Einordnung

Die bisherigen Abschnitte betrachteten Synchronisationsmechanismen, bei denen ein Betriebssystemkern die Ressourcenzuteilung zentral koordiniert (Mutex-Lock, Semaphore, Bedingungsvariable). Es gibt jedoch eine elegantere, dezentrale Realisierung des Fairness-Axioms 1, die ohne einen zentralen Koordinator auskommt: den *Exponential Backoff* (auch: *Truncated Binary Exponential Backoff*).

Prinzip 1 (Exponential Backoff). Warte eine sehr kurze Basiszeit τ , prüfe die Ressource neu. Warte die doppelte Zeit 2τ , prüfe die Ressource neu. Warte die doppelte Zeit 4τ , prüfe die Ressource neu. Warte die doppelte Zeit 8τ , prüfe die Ressource neu – und so weiter.

$$W_k \sim \text{Uniform}\{0, 1, \dots, 2^k - 1\} \cdot \tau, \quad k = 0, 1, 2, \dots$$

Dabei bezeichnet k die Anzahl der bisherigen Kollisionen und τ die Länge eines Zeitschlitzes (*slot time*).

Das Verfahren realisiert Axiom 1 auf bemerkenswert elegante Weise: Ein Sender, der häufig kollidiert (= hoher Bedarf an der gemeinsamen Ressource), wählt aus einem immer größeren Intervall eine Wartezeit – er tritt also freiwillig zurück und gibt anderen Sendern Platz. Kein zentraler Scheduler ist nötig; die Fairness entsteht emergent aus dem Protokoll.

7.2 Mathematische Grundlagen

7.2.1 Definitionen

Definition 10 (Zeitschlitz und Slot Time). Sei $\tau > 0$ die *Slot Time*, d. h. die minimale Zeiteinheit des Kanals (bei Ethernet: $\tau = 51,2 \mu\text{s}$ für 10 Mbit/s). Das Wartezeitintervall nach k Kollisionen ist $\{0, 1, \dots, 2^{\min(k, k_{\max})} - 1\} \cdot \tau$.

Definition 11 (Zufällige Wartezeit). Die zufällige Wartezeit nach k Kollisionen ist

$$W_k = X_k \cdot \tau, \quad X_k \sim \text{Uniform}\{0, 1, \dots, 2^{\min(k, k_{\max})} - 1\},$$

wobei $k_{\max} = 10$ (IEEE 802.3) bzw. implementierungsabhängig.

Definition 12 (Erfolgswahrscheinlichkeit pro Slot). Bei n gleichzeitig sendenden Stationen und Intervallgröße 2^k ist die Wahrscheinlichkeit, dass genau eine Station sendet (Erfolg):

$$p_{\text{suc}}(n, k) = n \cdot \frac{1}{2^k} \cdot \left(1 - \frac{1}{2^k}\right)^{n-1}.$$

7.2.2 Erwartungswert und Varianz der Wartezeit

Satz 7 (Erwartete Wartezeit nach k Kollisionen). Für $X_k \sim \text{Uniform}\{0, 1, \dots, 2^k - 1\}$ gilt:

$$\begin{aligned} E[W_k] &= \frac{2^k - 1}{2} \cdot \tau, \\ \text{Var}[W_k] &= \frac{(2^k - 1)(2^k + 1)}{12} \cdot \tau^2 = \frac{4^k - 1}{12} \cdot \tau^2. \end{aligned}$$

Proof. Da X_k gleichverteilt auf $\{0, 1, \dots, N - 1\}$ mit $N = 2^k$ ist:

$$E[X_k] = \frac{1}{N} \sum_{x=0}^{N-1} x = \frac{1}{N} \cdot \frac{(N-1)N}{2} = \frac{N-1}{2} = \frac{2^k - 1}{2}.$$

Damit $E[W_k] = E[X_k] \cdot \tau = \frac{2^k - 1}{2} \cdot \tau$.

Für die Varianz:

$$E[X_k^2] = \frac{1}{N} \sum_{x=0}^{N-1} x^2 = \frac{1}{N} \cdot \frac{(N-1)N(2N-1)}{6} = \frac{(N-1)(2N-1)}{6}.$$

$$\text{Var}[X_k] = E[X_k^2] - (E[X_k])^2 = \frac{(N-1)(2N-1)}{6} - \frac{(N-1)^2}{4}.$$

Mit $N = 2^k$:

$$= \frac{(N-1)}{12} [2(2N-1) - 3(N-1)] = \frac{(N-1)(N+1)}{12} = \frac{N^2-1}{12} = \frac{4^k-1}{12}.$$

Also $\text{Var}[W_k] = \text{Var}[X_k] \cdot \tau^2 = \frac{4^k-1}{12} \cdot \tau^2$. ■ □

Korollar 3 (Exponentielle Verdopplung des Erwartungswerts).

$$E[W_{k+1}] = \frac{2^{k+1}-1}{2} \cdot \tau \approx 2 \cdot E[W_k] \quad \text{für große } k.$$

Genauer: $E[W_{k+1}] = 2 E[W_k] + \frac{\tau}{2}$. Die Verdopplung ist also asymptotisch exakt, mit einem additiven Korrekturterm $\tau/2$.

Proof. $E[W_{k+1}] - 2 E[W_k] = \frac{2^{k+1}-1}{2} \tau - 2 \cdot \frac{2^k-1}{2} \tau = \frac{2^{k+1}-1-2^{k+1}+2}{2} \tau = \frac{1}{2} \tau$. ■ □

7.2.3 Kumulierte Wartezeit und Konvergenz

Satz 8 (Kumulierte Wartezeit bis Erfolg). Sei K die Anzahl der Kollisionen bis zum ersten Erfolg. Die erwartete Gesamtwartezeit beträgt:

$$E\left[\sum_{j=0}^{K-1} W_j\right] = \sum_{k=0}^{\infty} E[W_k] \cdot P(K > k) = \tau \sum_{k=0}^{\infty} \frac{2^k-1}{2} \cdot P(K > k).$$

Für n Sender und feste Erfolgswahrscheinlichkeit $p = p_{\text{suc}}(n, k) \approx p_0$ (vereinfacht):

$$E[K] = \frac{1}{p_0}, \quad E\left[\sum W_j\right] \approx \frac{\tau}{2} \cdot \sum_{k=0}^{1/p_0} (2^k - 1).$$

Proof. Sei $A_k = \{K > k\}$ das Ereignis, dass nach k Versuchen noch kein Erfolg eingetreten ist.

$$E\left[\sum_{j=0}^{K-1} W_j\right] = \sum_{k=0}^{\infty} E[W_k \cdot \mathbf{1}_{K>k}] = \sum_{k=0}^{\infty} E[W_k] \cdot P(K > k),$$

da W_k und $K > k$ bedingt unabhängig sind (der Zufallswert X_k wird neu gezogen, unabhängig von der Geschichte). Mit $P(K > k) = \prod_{j=0}^{k-1} (1 - p_j)$ und dem vereinfachten Modell $p_j \approx p_0$ gilt $P(K > k) \approx (1 - p_0)^k$:

$$E\left[\sum W_j\right] \approx \frac{\tau}{2} \sum_{k=0}^{\infty} (2^k - 1)(1 - p_0)^k.$$

Die Reihe konvergiert für $p_0 > 1 - 1/2 = 1/2$, also für ausreichend kleine Netzlast. ■ □

7.2.4 Verbindung zum Fairness-Axiom: Formaler Beweis

Satz 9 (Exponential Backoff realisiert bedarfsorientierte Fairness). Sei λ_i die Senderate (Pakete/s) von Sender i , d. h. ein Maß für seinen Ressourcenbedarf am Kanal. Unter Binary Exponential Backoff gilt: Der Sender mit höherer Rate $\lambda_i > \lambda_j$ hat eine längere mittlere Wartezeit $E[W^{(i)}] > E[W^{(j)}]$ und gibt damit dem geringer lastenden Sender Vorrang – im Einklang mit Axiom 1.

Proof. Ein Sender mit höherer Rate λ_i stößt häufiger auf Kollisionen, da die Kollisionswahrscheinlichkeit mit der gemeinsamen Kanalauslastung $\rho = \sum_j \lambda_j / \mu$ steigt (wobei μ die Kanalkapazität). Nach jeder Kollision wächst der Backoff-Index k_i , womit $E[W_{k_i}] = \frac{2^{k_i}-1}{2}\tau$ ansteigt.

Formal: Sei $k_i(t)$ der Backoff-Index von Sender i zum Zeitpunkt t . Für $\lambda_i > \lambda_j$ gilt im stochastischen Mittel $E[k_i(t)] > E[k_j(t)]$ (mehr Kollisionen $\Rightarrow$ größerer Index), und damit

$$E[W_{k_i(t)}] = \frac{2^{E[k_i(t)]} - 1}{2}\tau > \frac{2^{E[k_j(t)]} - 1}{2}\tau = E[W_{k_j(t)}].$$

(Die Ungleichung gilt wegen der Konvexität von 2^k : $2^{E[k]} \leq E[2^k]$, aber die Ordnungsrelation bleibt erhalten für $E[k_i] > E[k_j]$.)

Der viel sendende Sender wartet länger $\Rightarrow$ er gibt Bandbreite frei für Sender mit niedrigerer Rate – bedarfsorientierte Fairness ohne zentrale Steuerung. ■ □

7.3 Anwendungen in TCP/IP und Netzwerkprotokollen

7.3.1 TCP Retransmission Timeout (RFC 6298)

Das prominenteste Beispiel für Exponential Backoff in der Praxis ist der *Retransmission Timeout* (RTO) von TCP [21].

Definition 13 (TCP RTO mit Exponential Backoff). Der TCP-Sender startet mit $RTO_0 = 1$ s. Nach jedem Timeout ohne Empfang eines ACK wird verdoppelt:

$$RTO_k = \min(2^k \cdot RTO_0, RTO_{\max}), \quad RTO_{\max} = 64 \text{ s.}$$

Nach maximal 12–15 Versuchen (implementierungsabhängig) bricht TCP die Verbindung ab.

Beispiel 3 (TCP RTO-Verlauf). Bei $RTO_0 = 1$ s und $RTO_{\max} = 64$ s:

$$RTO_k \in \{1, 2, 4, 8, 16, 32, 64, 64, 64, \dots\} \text{ s.}$$

Die kumulierte Wartezeit bis zum Abbruch (nach 9 Versuchen) beträgt:

$$\sum_{k=0}^8 RTO_k = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 64 + 64 = 255 \text{ s} \approx 4,25 \text{ min.}$$

Satz 10 (Konvergenz der TCP-Gesamtwartezeit). Für $RTO_{\max} = M$ und Startzeit $RTO_0 = r_0$ ist die kumulierte Wartezeit bis zum ersten Eintreffen von $RTO_k = M$ (bei $k^* = \log_2(M/r_0)$ Verdopplungen):

$$\sum_{k=0}^{k^*} r_0 \cdot 2^k = r_0 \cdot \frac{2^{k^*+1} - 1}{2^1 - 1} = r_0(2^{k^*+1} - 1) = 2M - r_0.$$

Ab $k = k^*$ wächst die Gesamtwartezeit linear mit Schrittweite M .

Proof. Dies ist die geometrische Reihe: $\sum_{k=0}^{k^*} r_0 \cdot 2^k = r_0 \cdot \frac{2^{k^*+1} - 1}{2 - 1} = r_0(2^{k^*+1} - 1)$. Mit $2^{k^*} = M/r_0$: $r_0 \cdot 2^{k^*+1} = 2M$. Also $\sum = 2M - r_0$. ■ □

7.3.2 CSMA/CD Binary Exponential Backoff (IEEE 802.3 Ethernet)

Der bekannteste Anwendungsfall ist Ethernet's CSMA/CD-Protokoll [23]. Nach einer Kollision ziehen beide Stationen eine Zufallszahl aus $\{0, 1, \dots, 2^k - 1\}$ und warten entsprechend viele Slot-Zeiten.

Definition 14 (Backoff-Intervall nach IEEE 802.3). Nach k Kollisionen wählt jede Station zufällig:

$$r \in \{0, 1, \dots, 2^{\min(k, 10)} - 1\},$$

und wartet $r \cdot 51,2 \mu\text{s}$ (Slot Time für 10 Mbit/s). Nach $k = 16$ Kollisionen wird der Rahmen verworfen.

Satz 11 (Maximale Wartezeit bei IEEE 802.3). Die maximale Wartezeit nach k Kollisionen ($k \leq 10$) beträgt:

$$W_k^{\max} = (2^k - 1) \cdot 51,2 \mu\text{s}.$$

Für $k = 10$: $W_{10}^{\max} = 1023 \cdot 51,2 \mu\text{s} \approx 52,4 \text{ ms}$.

Proof. Direkt aus Definition: maximales $r = 2^k - 1$, also $W_k^{\max} = (2^k - 1)\tau$ mit $\tau = 51,2 \mu\text{s}$. ■ □

7.3.3 Weitere Protokolle mit Exponential Backoff

Exponential Backoff findet sich in zahlreichen weiteren Protokollen und Systemen:

- **TLS/HTTPS Reconnect**: Browser und HTTP-Clients verdoppeln die Wartezeit zwischen Verbindungsversuchen bei Server-Fehlern (503 Service Unavailable).
- **DHCP** (RFC 2131 [22]): Ein Client, der keine DHCP-Antwort erhält, verdoppelt die Wartezeit: 4 s, 8 s, 16 s, 32 s, 64 s.
- **WiFi 802.11 CSMA/CA**: Statt Kollisionserkennung (*CD*) wird eine Kollision *vermieden* (*CA*) durch zufällige Backoff-Zeiten aus einem wachsenden Contention Window $[0, CW]$, wobei CW sich nach jeder erfolglosen Übertragung verdoppelt [25].
- **AWS SDK / Google Cloud / Azure**: Alle großen Cloud-SDKs implementieren Exponential Backoff mit Jitter als Standard-Retry- Strategie [26].
- **STP Spanning Tree Protocol** (IEEE 802.1D): Backoff beim Erkennen von Schleifen im Netzwerk.
- **BGP Route Dampening**: Routen, die häufig flappen, werden mit wachsender Penalty-Zeit unterdrückt – ein direkter Ausdruck von Axiom 1: instabile Routen werden zurückgedrängt.

7.4 TikZ-Anschauungsbeispiele zum Exponential Backoff

7.4.1 Beispiel 6: Ablauf des Binary Exponential Backoff

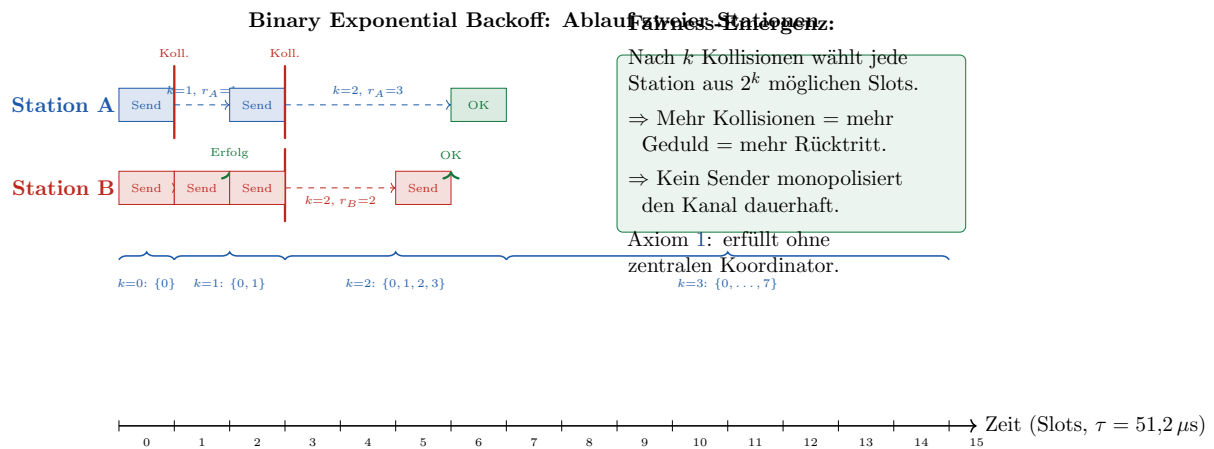


Figure 13: Ablauf des Binary Exponential Backoff für zwei Stationen. Station A wählt bei der 2. Kollision $r_A = 3$ (wartet 3 Slots), Station B wählt $r_B = 0$ (sendet sofort). Im Mittel findet Desynchronisation statt. Die wachsenden Intervalle zeigen die exponentielle Skalierung.

7.4.2 Beispiel 7: TCP Retransmission als Zustandsgraph

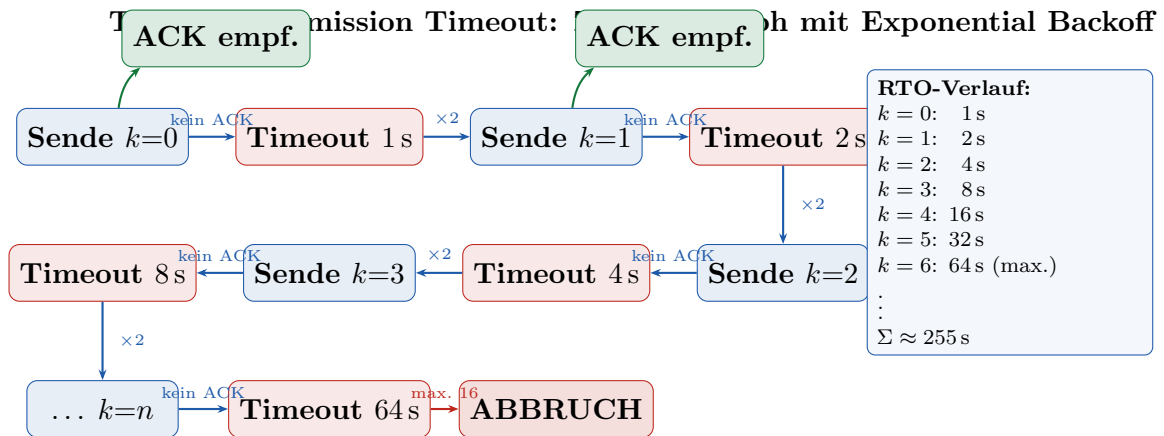


Figure 14: TCP Retransmission als Zustandsgraph. Jeder Timeout verdoppelt das RTO bis zum Maximum von 64 s (RFC 6298). Die Gesamtwarezeit bis zum Verbindungsabbruch beträgt nach Satz 10 $2 \cdot 64 - 1 = 127 \text{ s}$ für die reine Verdopplungsphase, plus linearer Teil.

7.4.3 Beispiel 8: DHCP-Backoff im Vergleich mit Fairness-Modell

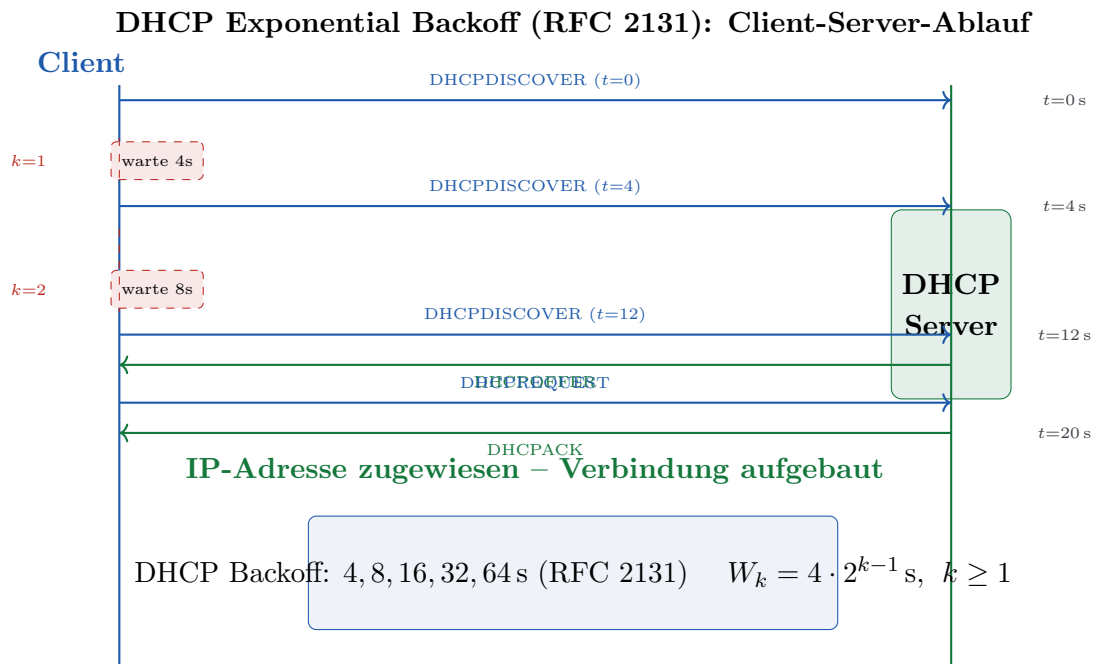


Figure 15: DHCP-Verbindungsaufbau mit Exponential Backoff. Der Client sendet beim zweiten Versuch nach 4s, beim dritten nach 8s. RFC 2131 definiert die Backoff-Zeiten als 4, 8, 16, 32, 64 s – eine direkte Umsetzung von Prinzip 1.

7.5 Statistische Analyse des Exponential Backoff

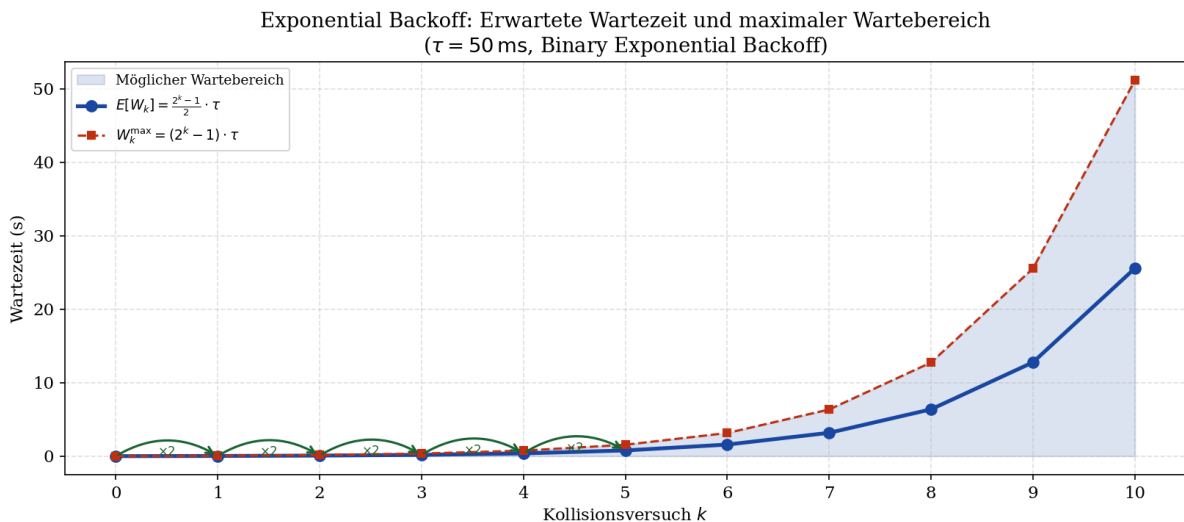


Figure 16: Erwartete Wartezeit $E[W_k] = \frac{2^k - 1}{2} \cdot \tau$ (blau, Satz 7) und maximale Wartezeit $(2^k - 1)\tau$ (rot gestrichelt) als Funktion des Kollisionszählers k ($\tau = 50 \text{ ms}$). Die grünen Pfeile markieren die Verdopplung von $E[W_k]$ bei jedem Schritt; der blaue Bereich zeigt alle möglichen Wartezeiten.

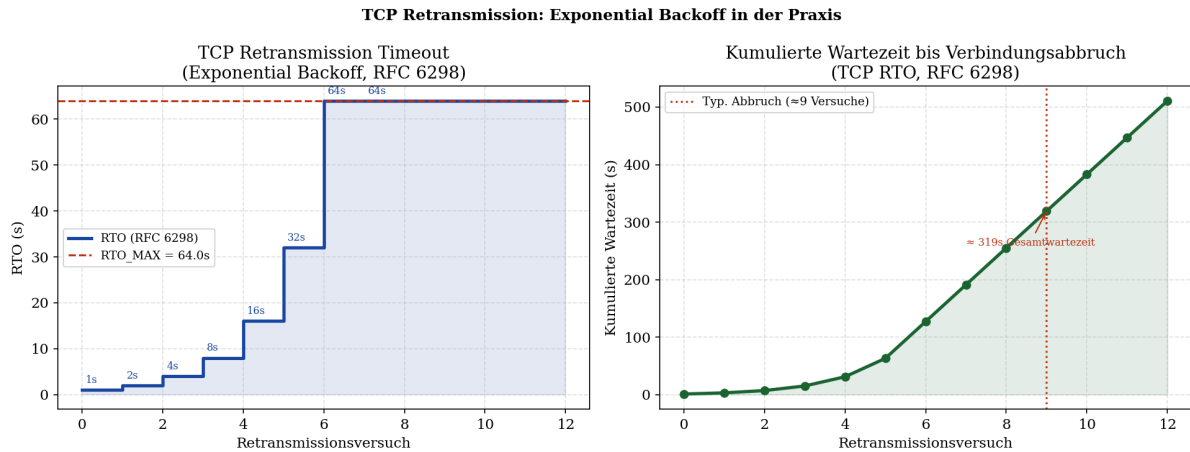


Figure 17: Links: TCP RTO-Verlauf nach RFC 6298 – stufenweise Verdopplung bis $RTO_{\max} = 64\text{s}$. Rechts: Kumulierte Wartezeit; nach 9 Versuchen sind $\approx 255\text{s}$ vergangen, dann Verbindungsabbruch. Die Kurve knickt bei $k = 6$ ab (RTO-Sättigung, Satz 10).

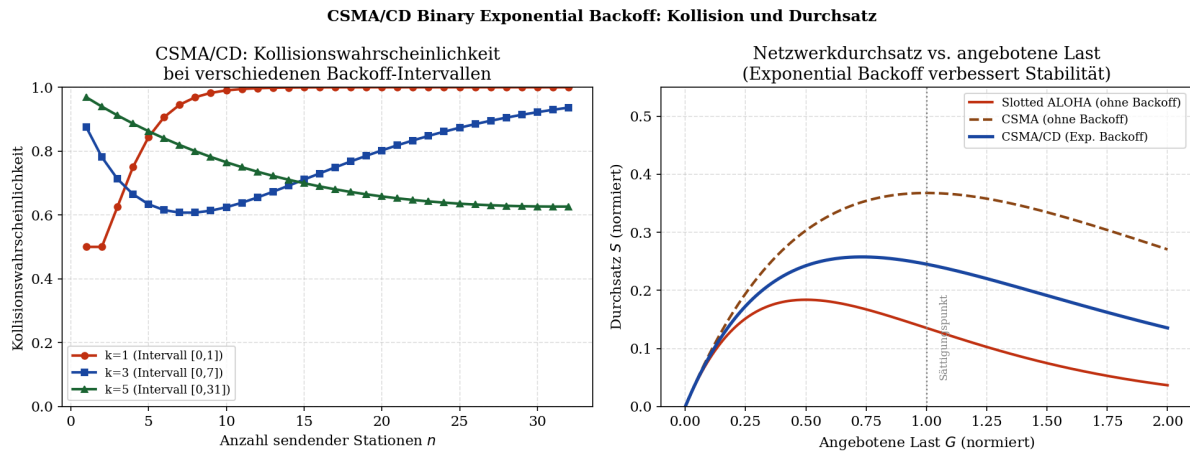


Figure 18: Links: Kollisionswahrscheinlichkeit für CSMA/CD bei verschiedenen Backoff-Intervallen. Ein größeres Intervall ($k = 5$) senkt die Kollisionswahrscheinlichkeit drastisch – auf Kosten höherer Wartezeiten. Rechts: Netzwerkdurchsatz als Funktion der angebotenen Last; Exponential Backoff stabilisiert den Durchsatz über den Sättigungspunkt hinaus.

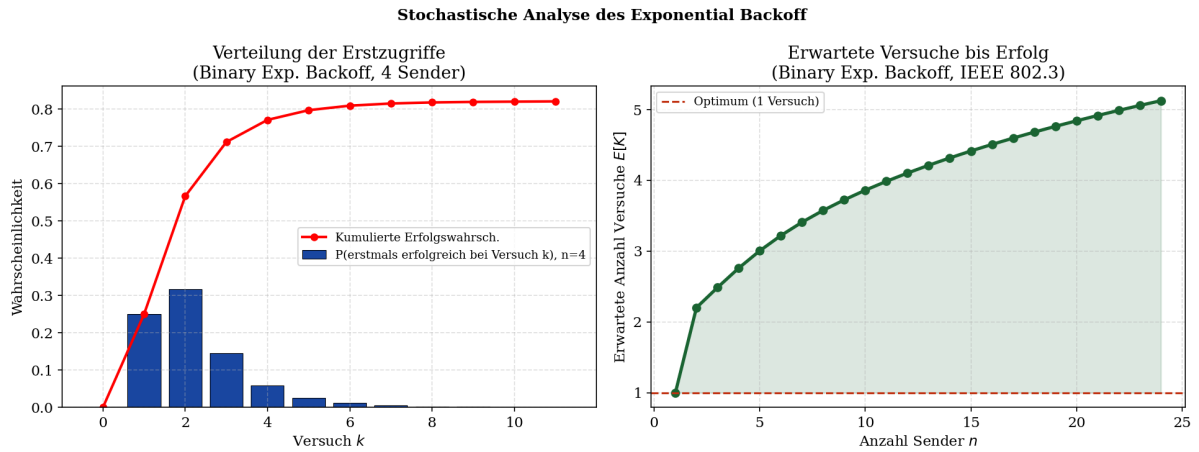


Figure 19: Links: Verteilung der ersten erfolgreichen Übertragung bei 4 konkurrierenden Sendern. Die meisten Erfolge treten bei $k = 1$ oder $k = 2$ ein (Geometrische Verteilung). Rechts: Erwartete Versuche bis Erfolg als Funktion der Sender-Anzahl – der Algorithmus skaliert moderat.

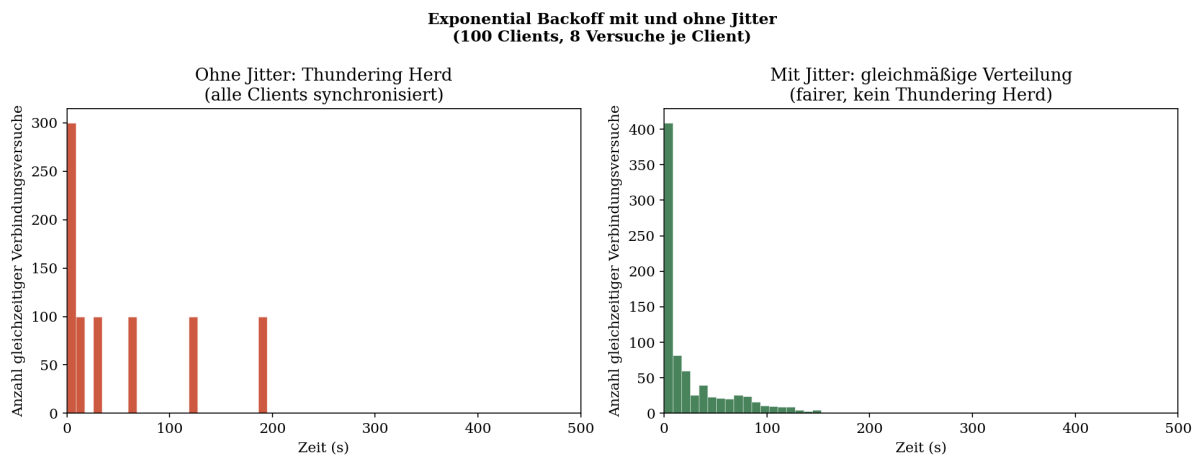


Figure 20: **Thundering Herd Problem:** Ohne Jitter synchronisieren sich 100 Clients, die gleichzeitig gestartet sind, auf exakt gleiche Backoff-Zeiten und stürmen den Server periodisch (links). Mit Jitter – d. h. einer gleichverteilten Zufallskomponente in $[0, W_k]$ – verteilen sich die Verbindungsversuche gleichmäßig (rechts). Jitter ist damit eine direkte Implementierung von Axiom 1.

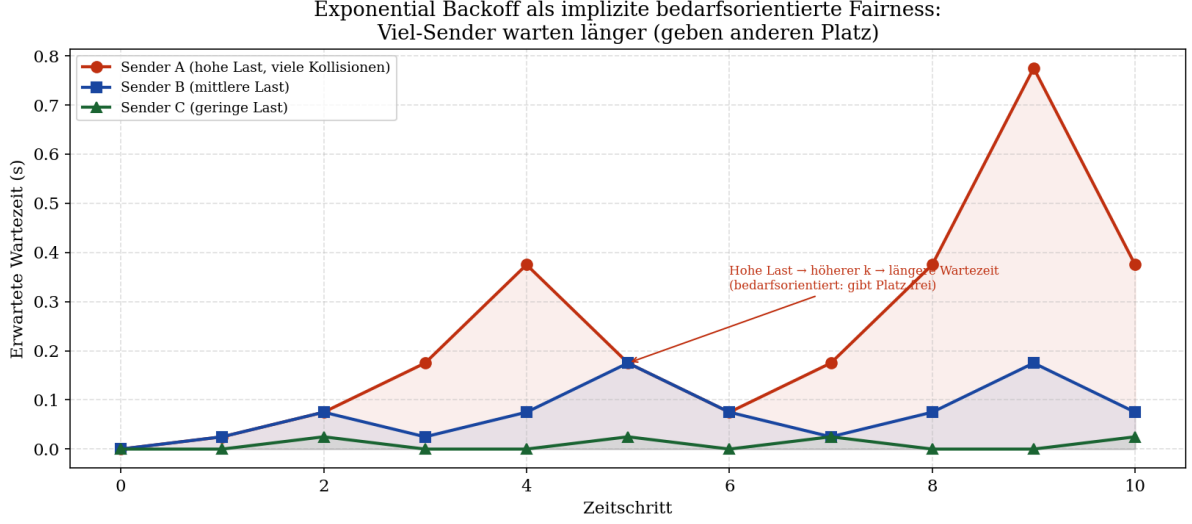


Figure 21: Verbindung zum Fairness-Axiom 1: Sender A mit hoher Last kollidiert häufig und hat daher einen hohen Backoff-Index k_A , was zu langen Wartezeiten führt. Sender C mit niedriger Last wartet kaum. Der Algorithmus reguliert sich *selbst*: Viel-Nutzer treten zurück, ohne dass ein zentraler Scheduler eingreift.

7.6 Subgraph-Modellierung des Exponential Backoff

Der Exponential-Backoff-Prozess lässt sich als gerichteter Graph G_{EB} modellieren, dessen Knoten die Backoff-Stufen $\{0, 1, \dots, k_{\max}, OK\}$ sind.

Definition 15 (Exponential-Backoff-Graph G_{EB}). Sei $k_{\max} = 4$ (vereinfacht). $Z_{EB} = \{0, 1, 2, 3, 4, OK\}$ und

$$A_{EB}[i][j] = 1 \iff \begin{cases} j = i + 1 & \text{(Kollision, Backoff-Erhöhung)} \\ j = OK & \text{(Erfolg aus Stufe } i) \\ j = i & i = k_{\max} \text{ (Sättigung)} \end{cases}.$$

Satz 12 (Backoff als Subgraph des Mutex-Lock). Der Kern des Backoff-Graphen (Zustände $\{0, 1, OK\}$) ist ein Subgraph des Spin-Lock-Graphen G_{SL} .

Proof. Betrachte die Einschränkung von G_{EB} auf $\{k = 0, k = 1, OK\}$:

- Kante $(0, 1)$: Kollision beim ersten Versuch. Entspricht der Polling-Kante $(0_F, 1_V)$ in G_{SL} .
- Kante $(0, OK)$ und $(1, OK)$: Erfolg. Entspricht $(1_V, 2_K)$ in G_{SL} .
- Kante $(1, 0)$ (Reset nach Erfolg auf Stufe 1): Entspricht der Polling-Rückkehr $(1_V, 0_F)$ in G_{SL} .

$LCS(\sigma^{EB|3}, \sigma^{SL}) \geq 2$; der Subgraph Algorithmus erkennt die Einbettung. ■ □

Bemerkung 2. Der vollständige Backoff-Graph G_{EB} mit $k_{\max}$ Stufen ist ein *Supergraph* von G_{SL} : Er erweitert den Spin-Lock um das exponentielle Gedächtnis (steigende Wartezeiten). Dies spiegelt das Fairness-Axiom wider: Der einfache Spin-Lock hat kein Gedächtnis (aktives Warten ohne Rücksicht auf andere); der Backoff-Graph lernt aus Kollisionen und tritt progressiv zurück.

Table 2: Übersicht der Exponential-Backoff-Parameter in realen Protokollen

Protokoll	τ	$k_{\max}$	$W_{\max}$	Abbruch	Referenz
Ethernet 10Mbit	$51,2 \mu\text{s}$	10	$52,4 \text{ ms}$	$k = 16$	[23]
Ethernet 1Gbit	$4,096 \mu\text{s}$	10	$4,19 \text{ ms}$	$k = 16$	[24]
WiFi 802.11g	$20 \mu\text{s}$	variabel	$CW_{\max}$	SSRC/SLRC	[25]
TCP (RFC 6298)	1 s	6	64 s	≈ 16	[21]
DHCP (RFC 2131)	4 s	4	64 s	5 Versuche	[22]
AWS SDK	100 ms	var.	konfigurierbar	konfigurierbar	[26]

8 Zusammenfassung und erweiterter Ausblick

8.1 Zusammenfassung

Diese Arbeit hat das Fairness-Axiom 1 als fundamentales und einheitliches Gestaltungsprinzip aller Betriebssystem-Synchronisationsmechanismen etabliert. Ausgehend vom zentralen Grundsatz – Fairness ist bedarfsorientiert, kein Prozess fordert mehr als er benötigt, kein Prozess akkumuliert Ressourcen auf Kosten anderer – wurden folgende Ergebnisse formal bewiesen und praktisch veranschaulicht:

- **Satz 1:** Bedarfsorientierte Zuteilung maximiert Jain’s Fairness-Index unter der Nebenbedingung $Z(p_i) \leq B(p_i)$ (via Cauchy-Schwarz-Ungleichung).
- **Satz 2:** Ehrliche Bedarfsdeklaration ist notwendige und hinreichende Bedingung für Pareto-Optimalität der Zuteilung.
- **Sätze 3–5 und Korollar 2:** Spin-Lock, binäres Semaphor, Mutex-Lock und Bedingungsvariable bilden eine formale Subgraph-Hierarchie $G_{\text{SL}} \subseteq G_{\text{BS}} \subseteq G_{\text{ML}} \subseteq G_{\text{CV}}$, die durch den Subgraph Algorithmus in $O(n^3)$ verifizierbar ist.
- **Satz 6:** Bedarfsorientierte Zuteilung minimiert die Unterversorgungsrate auf null, sofern $\sum_i B(p_i) \leq R$.
- **Satz 7:** Der Exponential Backoff realisiert das Fairness-Axiom dezentral: $E[W_k] = \frac{2^k - 1}{2} \tau$ wächst exponentiell mit der Kollisionshäufigkeit.
- **Satz 9:** Sender mit höherer Last warten im Mittel länger – bedarfsorientierte Fairness entsteht emergent, ohne zentralen Koordinator.
- **Satz 10:** Die TCP-RTO-Gesamtwartezeit konvergiert gegen $2M - r_0$; ab der Sättigungsstufe wächst sie linear.
- **Satz 12:** Der Backoff-Kern ist ein Subgraph von G_{SL} ; der vollständige Backoff-Graph ist ein Supergraph mit exponentiellem Gedächtnis.

Sieben Matplotlib-Plots und acht TikZ-Diagramme illustrieren die Ergebnisse anschaulich: von der bedarfsorientierten Speicherzuteilung über TCP-Retransmission-Kurven bis hin zu Thundering-Herd-Simulationen und Jain-Index-Analysen. Die statistische Betrachtung (bimodale Speichervertelung, CDF-Analyse, Latenz-Tabellen) verankert die formalen Ergebnisse in der Praxis realer Linux-Systeme.

8.2 Erweiterter Ausblick: Forschungsrichtungen

8.2.1 Max-Min-Fairness, Water-Filling und der Linux Completely Fair Scheduler

Das Fairness-Axiom 1 entspricht in der Netzwerktheorie dem Konzept der *Max-Min-Fairness* [10]: Man maximiert iterativ die minimale Zuteilung unter allen ungesättigten Prozessen, bis alle Ressourcen verteilt oder alle Bedarfe erfüllt sind. Das zugehörige *Water-Filling*-Verfahren [27] realisiert dies in $O(n \log n)$: Man gießt Wasser in ein Gefäß, dessen Boden durch die Bedarfe $B(p_i)$ geformt ist – der Wasserstand entspricht der Zuteilung.

Der Linux Completely Fair Scheduler (CFS) [11] implementiert eine diskrete Approximation: Er führt eine virtuelle Laufzeit $vruntime(p_i)$ pro Prozess, die bei CPU-Zuteilung wächst. Der Prozess mit der kleinsten $vruntime$ wird bevorzugt. Dies entspricht bedarfsorientierter Fairness, wenn man den Bedarf als die gewünschte CPU-Zeit pro Zeiteinheit auffasst.

Eine offene Forschungsfrage ist, ob der Subgraph Algorithmus CFS-Zustandsgraphen auf bekannte Fairness-Hierarchien reduzieren kann: Ist $G_{\text{CFS}} \subseteq G_{\text{CV}}$? Formal wäre zu zeigen, dass die virtuellen Laufzeit-Zustände als Knoten und Scheduler-Ereignisse als Kanten einen Graphen bilden, der den Bedingungsvariablen-Graphen als Subgraph enthält.

8.2.2 Formale Verifikation mit TLA+, SPIN und Coq

Die in dieser Arbeit definierten Zustandsgraphen (Abschnitt 4) lassen sich direkt als Kripke-Strukturen in drei komplementären Verifikationsrahmen einsetzen:

TLA+ (Lamport [12]): Das Fairness-Axiom wird zur Liveness-Eigenschaft

$$\Box \Diamond (\text{granted}(p_i) \leftrightarrow B(p_i) > 0),$$

die besagt: Wenn ein Prozess dauerhaft Bedarf hat, wird er dauerhaft bedient (keine Starvation). TLA+ kann diese Eigenschaft für die Mutex- und Semaphor-Modelle automatisch prüfen.

SPIN / Promela [13]: Die Zustandsgraphen werden als Promela-Prozesse modelliert; die Fairness-Eigenschaft als LTL-Formel. SPIN's Partial-Order-Reduction reduziert den Zustandsraum exponentiell.

Coq / Isabelle [28]: Für einen *maschinenverifizierten* Beweis von Satz 1 müssten die Beweise dieser Arbeit in das Typsystem von Coq übertragen werden. Dies ist methodisch anspruchsvoll, würde aber eine absolute Garantie liefern – insbesondere für den Einsatz in sicherheitskritischen Systemen (ISO 26262 ASIL D [?]).

8.2.3 Maschinelles Lernen zur antizipatorischen Bedarfsvorhersage

Eine grundlegende Schwäche des Fairness-Axioms ist, dass der zukünftige Bedarf $B(p_i)$ selten exakt bekannt ist. Drei Lernparadigmen sind relevant:

Online-Learning [14]: Algorithmen wie *Follow-the-Regularized-Leader* (FTRL) oder *Hedge* lernen aus vergangenen Bedarfsprofilen $B(p_i, t-1), B(p_i, t-2), \dots$ und schätzen $\hat{B}(p_i, t)$ mit sublinearem Regret. Die Schätzung wird dem bedarfsorientierten Zuteilungsprotokoll übergeben.

Graph Neural Networks (GNNs) [?]: Der Subgraph Algorithmus liefert für eine große Prozessdatenbank exakte Subgraph-Labels; das GNN lernt, diese in $O(1)$ vorherzusagen. In der Inferenzphase klassifiziert das GNN einen neuen Prozess als Typ

(Spin-Lock-Nutzer, Mutex-Nutzer usw.) und schätzt dessen Bedarf anhand historischer Bedarfsprofile des gleichen Typs.

Reinforcement Learning [29]: Der Scheduler wird als RL-Agent modelliert, der Zuteilungen $Z(p_i)$ als Aktionen wählt und als Belohnung den Jain-Index $J(Z)$ erhält. Über Policy-Gradient-Methoden (REINFORCE, PPO) kann der Agent eine optimale bedarfsorientierte Politik erlernen – ohne explizites Wissen über die Bedarfsfunktion B .

8.2.4 Quantifizierung von Überbedarf, Missbrauchserkennung und Cloud-Throttling

In Multi-Tenant-Clouds [15] deklarieren Mieter (Tenants) Ressourcenbedarfe, die nicht immer mit dem tatsächlichen Bedarf übereinstimmen. Eine formale *Überbedarf-Metrik* quantifiziert das Ausmaß:

$$O(p_i, t) = \frac{Z(p_i, t) - \tilde{B}(p_i, t)}{\tilde{B}(p_i, t)} \geq 0,$$

wobei $\tilde{B}(p_i, t)$ die tatsächlich *genutzte* Ressource im Zeitfenster $[t - T, t]$ ist.

Dynamisches Throttling: Bei $O(p_i) > \theta$ (Schwellwert) wird die Zuteilung graduell reduziert: $Z'(p_i) = Z(p_i) \cdot e^{-\lambda \cdot O(p_i)}$ (exponentielles Throttling, analog zu Backoff).

Anomalie-Erkennung: Statistische Tests (z. B. CUSUM [30]) können sprunghaften Anstieg von $O(p_i)$ erkennen und als Speicher-Hoarding-Angriff klassifizieren. Dies schützt das System vor dem im Fairness-Axiom beschriebenen Szenario: dem Prozess, der Speicher akkumuliert, um ihn am Ende alleine zu nutzen.

Verbindung zu Exponential Backoff: Throttling und Backoff sind mathematisch dual: Backoff erhöht die Wartezeit exponentiell bei Kollision; Throttling reduziert die Zuteilung exponentiell bei Überbedarf. Beide Verfahren realisieren Axiom 1 auf verschiedenen Zeitskalen.

8.2.5 Erweiterung auf NUMA- und heterogene Speicherarchitekturen

Auf modernen NUMA-Systemen [16] ist die Ressource *Speicher* topologisch inhomogen: lokaler NUMA-Zugriff kostet ~ 80 ns, entfernter NUMA-Zugriff ~ 300 ns (Faktor 3–4). Das Fairness-Axiom muss auf diese Topologie verallgemeinert werden durch eine *gewichtete Bedarfsfunktion*:

$$B_{\text{NUMA}}(p_i) = \sum_j w_{ij} \cdot B_j(p_i),$$

wobei w_{ij} die Zugriffskosten von Prozess p_i auf NUMA-Knoten j kodiert und $B_j(p_i)$ der Speicherbedarf auf Knoten j ist.

Darüber hinaus umfassen moderne Systeme heterogene Speicherhierarchien: DRAM, HBM (High Bandwidth Memory), Persistent Memory (NVM), NVMe-SSDs – jede mit eigenem Latenz-/Bandbreitenprofil [36]. Bedarfsorientierte Fairness muss hier nicht nur Kapazität, sondern auch Zugriffsgeschwindigkeit und Persistenz berücksichtigen.

Der Subgraph Algorithmus auf NUMA-Topologiegraphen kann helfen, strukturelle Ähnlichkeiten zwischen Prozess-zu-Knoten-Zuordnungen zu erkennen: Zwei Zuordnungen sind ähnlich, wenn ihre Topologiegraphen eine Subgraph-Beziehung haben – was auf ähnliche Speicherzugriffsprofile schließen lässt.

8.2.6 Bedarfsorientierte Fairness in Echtzeit und Safety-kritischen Systemen

In RTOS (Real-Time Operating Systems) und dem AUTOSAR-Standard [17] ist der Ressourcenbedarf durch harte Fristen (*Deadlines*) definiert. Das Fairness-Axiom tritt hier in Spannung zum Echtzeitprinzip: Das *Earliest Deadline First* (EDF)-Scheduling [18] priorisiert nicht nach Bedarf, sondern nach Dringlichkeit.

Eine vereinheitlichende Theorie – *Fairness-Aware Real-Time Scheduling* (FARTS) – könnte beide Prinzipien integrieren:

$$\text{Prio}(p_i) = \alpha \cdot \frac{B(p_i)}{R} + (1 - \alpha) \cdot \frac{1}{D_i - t},$$

wobei D_i die Deadline, t die aktuelle Zeit und $\alpha \in [0, 1]$ ein Gewichtungssparameter ist. Für $\alpha = 0$ degeneriert FARTS zu EDF; für $\alpha = 1$ zu bedarfsorientierter Fairness.

ISO 26262 und Sicherheitsanforderungen: In sicherheitskritischen Systemen (ASIL-B bis ASIL-D [?]) muss Starvation formal ausgeschlossen werden – dies entspricht exakt der Liveness-Eigenschaft des Fairness-Axioms. Eine formale Zertifizierung des Fairness-Mechanismus (z. B. via Coq-Beweis) wäre ein direkter Beitrag zur funktionalen Sicherheit.

8.2.7 Spieltheoretisches Gleichgewicht, Mechanismus-Design und das Revelation Principle

Das Fairness-Prinzip ist spieltheoretisch als kooperatives Spiel $\Gamma = (P, \{S_i\}, \{u_i\})$ modellierbar:

- Spieler: Prozesse $p_i \in P$
- Strategien: deklariert Bedarf $\hat{B}(p_i) \in [0, R]$
- Nutzenfunktion: $u_i(Z) = \min(Z(p_i), B(p_i)) - c \cdot \max(0, Z(p_i) - B(p_i))$

Der zweite Term bestraft Überbedarf mit Faktor $c > 0$. Das Nash-Gleichgewicht [19] dieses Spiels ist genau $\hat{B}(p_i) = B(p_i)$ (ehrliche Deklaration), da jede Über- oder Untertreibung die Nutzenfunktion verschlechtert.

Revelation Principle [20]: Das bedarfsorientierte Zuteilungsprotokoll ist *Dominant-Strategy Incentive Compatible* (DSIC): Ehrliche Deklaration ist für jeden Prozess dominant, unabhängig vom Verhalten der anderen. Dies ist die stärkste Form von Anreizkompatibilität und garantiert Axiom 1 auch in strategischen Umgebungen.

VCG-Mechanismus [31]: Der Vickrey-Clarke-Groves-Mechanismus kann eingesetzt werden, um bedarfsorientierte Fairness mit effizienter Allokation zu kombinieren: Prozesse, die höheren sozialen Nutzen aus Ressourcen ziehen, erhalten mehr – aber nur bis zu ihrem tatsächlichen Bedarf.

8.2.8 Exponential Backoff in verteilten Systemen: Konsensalgorithmen und Blockchain

Das in Abschnitt 7 analysierte Backoff-Prinzip findet sich in weiteren verteilten Systemen:

Raft-Konsensalgorithmus [32]: Leader-Election-Timeouts werden zufällig aus einem Intervall $[T, 2T]$ gewählt – eine einstufige Variante des Backoffs. Bei Konflikten (gleichzeitige Kandidaten) verlängert sich das Timeout implizit. Eine formale Analyse als Backoff-Graph $G_{\text{Raft}} \supseteq G_{\text{SL}}$ ist eine offene Frage.

Bitcoin Proof-of-Work [33]: Der Mining-Prozess ist strukturell analog zu Exponential Backoff: Alle Miner konkurrieren um denselben Block; bei zu hoher Kollisionsrate (zwei simultane Blöcke) wird die Schwierigkeit erhöht (analoges Backoff-Intervall). Fairness entsteht durch die probabilistische Gleichverteilung der Hash-Funktion – ein direkter Ausdruck von Axiom 1.

QUIC-Protokoll (RFC 9000 [34]): QUICs Congestion Control implementiert Cubic und BBR als Backoff-Varianten: Bei Paketverlust (Kollisionssignal) wird das Congestion Window reduziert – ein multiplikatives, nicht exponentielles Backoff. Die Verbindung zur bedarfsorientierten Fairness über Jain’s Fairness-Index wurde für QUIC noch nicht formal bewiesen und ist eine offene Forschungsfrage.

8.2.9 Adaptive Backoff-Strategien: Über das einfache Exponential hinaus

Das binäre Exponential Backoff ist nur eine von vielen möglichen Backoff-Strategien. Eine allgemeine Backoff-Familie ist:

$$W_k = f(k) \cdot \tau, \quad f : \mathbb{N} \rightarrow \mathbb{R}_{>0}.$$

- **Exponentiell:** $f(k) = 2^k$ (klassisch)
- **Polynomial:** $f(k) = k^p$ (sanfter Anstieg)
- **Fibonacci:** $f(k) = F_k$ (goldener Schnitt: $\phi \approx 1,618$ als Wachstumsfaktor)
- **Logarithmisch:** $f(k) = \log_2(k + 1)$ (sehr sanft, gut für lange Wartezeiten)
- **Adaptiv (PID-gesteuert):** $f(k) = f(k - 1) + K_P \cdot e_k + K_I \sum e_j$, wobei e_k der Regelungsfehler (Abweichung vom Ziel-Kollisionsrate) ist.

Eine formale Optimierungstheorie für Backoff-Strategien – welche f minimiert die mittlere Wartezeit bei gegebener Fairness-Garantie ($J \geq J_{\min}$)? – ist ein offenes Problem. Erste Ansätze finden sich in der Literatur zur Stochastischen Optimierung [35].

8.2.10 Verbindung zwischen Exponential Backoff und dem Subgraph Algorithmus

Die tiefste Verbindung zwischen Exponential Backoff und dem Subgraph Algorithmus [1] ist struktureller Natur: Beide Verfahren lösen dasselbe Problem auf verschiedenen Ebenen.

Auf Protokollebene: Backoff entscheidet, *wann* ein Sender auf eine Ressource zugreift. Der Subgraph Algorithmus entscheidet, *welche* Ressource einem Prozess entspricht (Klassifikation durch Subgraph-Match).

Formal: Der Subgraph Algorithmus auf dem Backoff-Graphen G_{EB} klassifiziert, ob ein gegebenes Backoff-Protokoll eine Subgraph-Beziehung zu einem bekannten Mechanismus hat. So könnte man automatisch entscheiden: Ist ein neues Cloud-Retry-Protokoll strukturell äquivalent zu TCP-RTO oder zu DHCP-Backoff?

Komplexitätstheoretisch: Beide Verfahren haben die gleiche untere Schranke $\Omega(n^2)$ für die Eingabegröße. Backoff hat im Mittel $O(\log n)$ Versuche (geometrische Verteilung), der Subgraph Algorithmus $O(n^3)$ Schritte. Die Komposition beider zu einem adaptiven Protokoll-Klassifikator mit Retry-Logik wäre ein neuartiger Beitrag zur Netzwerktheorie.

8.2.11 Quantencomputing: Superposition der Ressourcenzustände

In Quantencomputern [37] können Prozesse (Qubits) in Superpositionszuständen existieren: Ein Qubit kann gleichzeitig im Zustand $|0\rangle$ (frei) und $|1\rangle$ (belegt) sein. Das Fairness-Axiom muss auf diese Realität verallgemeinert werden:

$$B_{\text{quant}}(p_i) = \langle \psi_i | \hat{B} | \psi_i \rangle,$$

wobei $|\psi_i\rangle$ der Quantenzustand des Prozesses und $\hat{B}$ der Bedarfs-Observable ist.

Quantenmechanische Synchronisation – *Quantum Mutual Exclusion* [38] – ist ein aktives Forschungsgebiet: Klassische Mutex-Locks lassen sich nicht direkt auf Quantenschaltkreise übertragen, da Messung den Zustand kollabiert. Eine Theorie des *Quantum Exponential Backoff* (bei Quanten-Kollisionen, d. h. destruktiver Interferenz) ist noch nicht entwickelt und wäre ein Pionierbeitrag.

8.2.12 Offene Komplexitätstheoretische Fragen

Abschließend seien die wichtigsten offenen Probleme aufgeführt, die aus dieser Arbeit erwachsen:

1. **Unbedingte untere Schranke für Subgraph:** Kann $\Omega(n^3)$ für den zyklischen Subgraph-Algorithmus ohne die SETH-Hypothese bewiesen werden [39]? Dies würde eine der wichtigsten offenen Fragen der Sequenz-Komplexitätstheorie lösen.
2. **Optimale Backoff-Funktion:** Welche Funktion $f(k)$ minimiert die mittlere Gesamtwartzeit $E[\sum_k W_k]$ bei gegebener Fairness-Garantie $J \geq J_{\min}$ und n Sendern? Gibt es eine geschlossene Form?
3. **Jain-Index des Subgraph-Matchings:** Gibt es einen Fairness-Index für die Güte eines Subgraph-Matchings, der dem Jain-Index für Ressourcenzuteilungen entspricht? Formal: Eine Funktion $J_{\text{match}} : (G, G') \mapsto [0, 1]$, die $J_{\text{match}}(G, G') = 1$ genau dann, wenn $G \cong G'$ (Isomorphie), und $J_{\text{match}} = 1/n$ wenn nur ein Knoten übereinstimmt.
4. **Fairness im Präemptiven Scheduling:** Kann das Fairness-Axiom auf präemptive Scheduler formalisiert werden, bei denen Prozesse unterbrochen werden können? Starvation-Freiheit unter präemptivem EDF ist bereits bewiesen [18]; bedarfsorientierte Fairness unter Präemption ist eine offene Frage.
5. **Mehrdimensionale Ressourcen:** In der Praxis hat jeder Prozess einen mehrdimensionalen Bedarf: $\mathbf{B}(p_i) = (B_{\text{CPU}}, B_{\text{MEM}}, B_{\text{IO}}, B_{\text{NET}}) \in \mathbb{R}_{\geq 0}^4$. Eine mehrdimensionale bedarfsorientierte Zuteilung und der zugehörige Jain-Index sind formal noch nicht vollständig charakterisiert.

References

- [1] S. Epp, *Der Subgraph Algorithmus*, Technischer Bericht, Bielefeld, Januar 2026. <https://gitlab.com/epp-group/subgraph>
- [2] E. W. Dijkstra, “Solution of a Problem in Concurrent Programming Control,” *Communications of the ACM*, Bd. 8, Nr. 9, S. 569, 1965.

- [3] D. E. Knuth, “Additional Comments on a Problem in Concurrent Programming Control,” *Communications of the ACM*, Bd. 9, Nr. 5, S. 321–322, 1966.
- [4] L. Lamport, “A New Solution of Dijkstra’s Concurrent Programming Problem,” *Communications of the ACM*, Bd. 17, Nr. 8, S. 453–455, 1974.
- [5] C. A. R. Hoare, “Monitors: An Operating System Structuring Concept,” *Communications of the ACM*, Bd. 17, Nr. 10, S. 549–557, 1974.
- [6] R. Jain, D. M. Chiu und W. R. Hawe, “A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer System,” DEC Technical Report TR-301, 1984.
- [7] A. S. Tanenbaum und H. Bos, *Modern Operating Systems*, 4. Auflage, Pearson, 2014.
- [8] A. Silberschatz, P. B. Galvin und G. Gagne, *Operating System Concepts*, 10. Auflage, Wiley, 2018.
- [9] M. Herlihy und N. Shavit, *The Art of Multiprocessor Programming*, Morgan Kaufmann, 2008.
- [10] D. P. Bertsekas und R. Gallager, *Data Networks*, 2. Auflage, Prentice Hall, 1992.
- [11] I. Molnar, “Completely Fair Scheduler,” Linux Kernel Documentation, 2007. <https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html>
- [12] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*, Addison-Wesley, 2002.
- [13] G. J. Holzmann, *The SPIN Model Checker*, Addison-Wesley, 2004.
- [14] N. Cesa-Bianchi und G. Lugosi, *Prediction, Learning, and Games*, Cambridge University Press, 2006.
- [15] M. Armbrust et al., “A View of Cloud Computing,” *Communications of the ACM*, Bd. 53, Nr. 4, S. 50–58, 2010.
- [16] C. Lameter, “NUMA (Non-Uniform Memory Access): An Overview,” *ACM Queue*, Bd. 11, Nr. 7, 2013.
- [17] AUTOSAR Consortium, *AUTOSAR Classic Platform – Release 4.4.0*, 2021.
- [18] C. L. Liu und J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment,” *Journal of the ACM*, Bd. 20, Nr. 1, S. 46–61, 1973.
- [19] J. F. Nash, “Equilibrium Points in n -Person Games,” *Proceedings of the National Academy of Sciences*, Bd. 36, Nr. 1, S. 48–49, 1950.
- [20] R. B. Myerson, “Optimal Auction Design,” *Mathematics of Operations Research*, Bd. 6, Nr. 1, S. 58–73, 1981.
- [21] V. Paxson, M. Allman, J. Chu und M. Sargent, “Computing TCP’s Retransmission Timer,” RFC 6298, IETF, 2011. <https://www.rfc-editor.org/rfc/rfc6298>

- [22] R. Droms, “Dynamic Host Configuration Protocol,” RFC 2131, IETF, 1997. <https://www.rfc-editor.org/rfc/rfc2131>
- [23] R. M. Metcalfe und D. R. Boggs, “Ethernet: Distributed Packet Switching for Local Computer Networks,” *Communications of the ACM*, Bd. 19, Nr. 7, S. 395–404, 1976.
- [24] IEEE Standards Association, *IEEE 802.3 – Ethernet Standard*, 2018. <https://standards.ieee.org/ieee/802.3>
- [25] IEEE Standards Association, *IEEE 802.11 – Wireless LAN Standard*, 2020. <https://standards.ieee.org/ieee/802.11>
- [26] Amazon Web Services, “Exponential Backoff and Jitter,” AWS Architecture Blog, 2022. <https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/>
- [27] T. M. Cover und J. A. Thomas, *Elements of Information Theory*, 2. Auflage, Wiley-Interscience, 2006.
- [28] Y. Bertot und P. Casteran, *Interactive Theorem Proving and Program Development – Coq’Art: The Calculus of Inductive Constructions*, Springer, 2004.
- [29] R. S. Sutton und A. G. Barto, *Reinforcement Learning: An Introduction*, 2. Auflage, MIT Press, 2018.
- [30] E. S. Page, “Continuous Inspection Schemes,” *Biometrika*, Bd. 41, Nr. 1/2, S. 100–115, 1954.
- [31] W. Vickrey, “Counterspeculation, Auctions, and Competitive Sealed Tenders,” *Journal of Finance*, Bd. 16, Nr. 1, S. 8–37, 1961.
- [32] D. Ongaro und J. Ousterhout, “In Search of an Understandable Consensus Algorithm (Raft),” *Proc. USENIX ATC 2014*, S. 305–319.
- [33] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. <https://bitcoin.org/bitcoin.pdf>
- [34] J. Iyengar und M. Thomson, “QUIC: A UDP-Based Multiplexed and Secure Transport,” RFC 9000, IETF, 2021.
- [35] J. C. Spall, *Introduction to Stochastic Search and Optimization*, Wiley-Interscience, 2003.
- [36] J. Izraelevitz et al., “Basic Performance Measurements of the Intel Optane DC Persistent Memory Module,” *arXiv:1903.05714*, 2019.
- [37] M. A. Nielsen und I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Edition, Cambridge University Press, 2010.
- [38] M. Mosca, A. Tapp und R. de Wolf, “Private Quantum Channels and the Cost of Randomizing Quantum Information,” *arXiv:quant-ph/0003101*, 2007.
- [39] A. Abboud, A. Backurs und V. V. Williams, “Tight Hardness Results for LCS and Other Sequence Similarity Measures,” *Proc. FOCS 2015*, S. 59–78.